UNIVERSIDAD ABIERTA INTERAMERICANA

Facultad de Tecnología Informática

WardrobeFlow

Sistema de Gestión de Alquiler de Prendas por Suscripción

Trabajo Integrador: Ingeniería de Software

Materia	Ingeniería de Software
Localización	Centro
Comisión	3-A
Turno	Noche
Alumnas	Bolivar, Magali - Morana, Valentina
Docentes	Silva, Juan Ignacio - Avila, Javier Ramón
Año	2026

Historial de Revisión

Fecha	Versión	Autor	Descripción	Responsable de revisión	Correcciones
14/04/2026	1.0	Magali Bolivar - Valentina Morana	Creación inicial – Iteración 1: G00, T01, T02, T06a, T03	Juani	
16/04/2026	1.1	Magali Bolivar - Valentina Morana	Corrección de textos	Juani	
19/04/2026	1.2	Magali Bolivar - Valentina Morana	Definición de negocio (compra-venta). Corrección del rol Cliente como externo al sistema.	Juani	
23/04/2026	1.3	Magali Bolivar - Valentina Morana	Correcciones pre-entrega #1	Juani	
30/04/2026	1.4	Magali Bolivar - Valentina Morana	Correcciones pre-entrega #1.1	Juani	
11/06/2026	1.4	Magali Bolivar - Valentina Morana	Correcciones pre-entrega #2	Juani	

- Ampliar el Historial de Cambios para registrar qué dato se modificó y su valor anterior/nuevo sin afectar datos sensibles
- Centralizar el ABM de usuarios en un único panel de administración.
- Reflejar las correcciones realizadas dentro del Historial de Cambios.
- Mostrar un mensaje genérico ante errores inesperados ("Contacte al administrador").
- Incorporar filtros utilizando únicamente datos no sensibles (nombre, apellido, mail).
- Documentar el cambio de roles y su gestión de errores.

- Definir qué perfiles pueden visualizar y administrar los Dígitos Verificadores (DV).
- Especificar qué acciones se registran en los logs de auditoría y su relación con los DV.

ÍNDICE DE CONTENIDOS

Historial de Revisión 2
G00. Descripción Global del Producto 5
G01. Propósito 6
G02. Descripción Funcional y Alcance 6
G03. Definiciones, Acrónimos y Abreviaciones 8
Definiciones 8
Acrónimos y Abreviaciones 8
G04. Descripción de Participantes y Roles del Sistema 9
T01. Arquitectura Base 10
Diagrama de Componentes 11
Mapa de Navegación MDI 14
T02. Gestión de Login / Logout del Sistema 17
T02.1 Gestión de Login del Sistema 17
Descripción detallada de funcionamiento 17
Diagrama de Casos de Uso 18
Especificación de Caso de Uso 18
Diagrama de Clases 20
Diagrama de Secuencia 21
DER 22
T02.2 Gestión de Logout del Sistema 23
Diagrama de Caso de Uso 24
Especificación de Caso de Uso 24
Diagrama de Clases 26
Diagrama de Secuencia 26
DER 28
T03. Gestión de Encriptado 30
Objetivo 30
Descripción detallada de funcionamiento 30
Algoritmos Implementados – Hash de Contraseñas PBKDF2-SHA256 30

Algoritmos Implementados – Cifrado Simétrico AES-128-CBC 31
Flujo de aplicación por caso de uso 31
Descripción de métodos principales 32
T04. Gestión de Perfiles de Usuario 33
Objetivo 33
Descripción funcional 33
Explicación del patrón Composite 33
Diagrama de clases 33
Diagrama de secuencia 33
DER 33
T05. Gestión de Múltiples Idiomas 33
Objetivo 33
Descripción funcional 33
Explicación del patrón Observer 33
Diagrama de clases 34
Diagrama de secuencia 34
T06a. Gestión de Bitácora 34
Objetivo 34
Descripción detallada de funcionamiento 34
Descripción de métodos 36
Niveles de criticidad (BE.Criticidad) 36
Caso de Uso 37
Especificación de Caso de Uso 38
Diagrama de Clases 40
Diagrama de Secuencia 42
DER 43
T06b. Control de Cambios 43
Objetivo 43
Descripción funcional 43
Diagrama de clases 43
DER 43
Diagrama de secuencia 43
T08. Dígitos Verificadores 43

Objetivo 43
Descripción funcional 44
Algoritmo DVH/DVV 44
Diagrama de clases 44
DER 44
Diagrama de secuencia 44
N01. Especificación funcional por proceso de negocio 45
N01. Proceso de Negocio 1 – Gestión de Clientes y Suscripciones 45
Roles Intervinientes 45
Descripción funcional del proceso 46
Diagrama de Proceso 46
Diagrama de Clases 46
N02. Proceso de Negocio 2 – Gestión de Inventario de Prendas 46
Roles Intervinientes 46
Descripción funcional del proceso 46
Diagrama de Proceso 47
Diagrama de Clases 47
N03. Proceso de Negocio 3 – Gestión de Pedidos de Alquiler 47
Roles Intervinientes 47
Descripción funcional del proceso 47
Diagrama de Proceso 48
Diagrama de Clases 48
N04. Proceso de Negocio 4 – Gestión de Despachos 48
Roles Intervinientes 48
Descripción funcional del proceso 48
Diagrama de Proceso 48
Diagrama de Clases 48
N05. Proceso de Negocio 5 – Gestión de Devoluciones 49
Roles Intervinientes 49
Descripción funcional del proceso 49
Diagrama de Proceso 49
Diagrama de Clases 49
N06. Proceso de Negocio 6 – Gestión de Incidencias 49

Roles Intervinientes 50
Descripción funcional del proceso 50
Diagrama de Proceso 50
Diagrama de Clases 50
N07. Proceso de Negocio 7 – Compra Definitiva de Prenda 50
Roles Intervinientes 50
Descripción funcional del proceso 51
Diagrama de Proceso 51
Diagrama de Clases 51
N08. Proceso de Negocio 8 – Generación de Reportes Operativos 51
Roles Intervinientes 51
Descripción funcional del proceso 52
Diagrama de Proceso 52
Diagrama de Clases 52

G00. Descripción Global del Producto

G01. Propósito

WardrobeFlow surge como respuesta a la creciente demanda de consumo de moda de forma flexible y sustentable. La organización ofrece un servicio de alquiler de indumentaria por suscripción, mediante el cual los clientes acceden a un catálogo de prendas, seleccionan las que desean utilizar durante un período determinado y las devuelven al finalizar el ciclo. Este modelo promueve un consumo responsable y circular, permitiendo renovar el guardarropa de manera continua sin necesidad de compra definitiva.

El sistema WardrobeFlow tiene como propósito centralizar y automatizar la operación interna de la organización, reemplazando procesos manuales o fragmentados que impactan negativamente en la eficiencia operativa. En particular, resuelve problemáticas concretas como la falta de trazabilidad sobre el estado de las prendas en cada momento del ciclo, la dificultad para controlar la disponibilidad en tiempo real, la ausencia de mecanismos de auditoría sobre las operaciones realizadas por los usuarios del sistema y la imposibilidad de generar reportes operativos ágiles para la toma de decisiones.

Adicionalmente, el sistema contempla la posibilidad de que un cliente opte por adquirir definitivamente una prenda que tiene en alquiler, abonando el precio de compra correspondiente. En ese momento se registra una operación de compra-venta, se genera el comprobante correspondiente y la prenda se da de baja del catálogo activo. Esta funcionalidad es gestionada íntegramente por el personal interno; el cliente no accede al sistema en ningún momento.

G02. Descripción Funcional y Alcance

El sistema gestiona el ciclo completo del servicio de alquiler de prendas: desde el registro del cliente y la activación de su suscripción, hasta la devolución, inspección y eventual registro de incidencias.

Módulo	Descripción	Roles con acceso
M01 – Usuarios y Permisos	Ciclo de vida de usuarios, roles y permisos. Asignación dinámica de perfiles y habilitación de accesos según el rol del usuario.	Administrador
M02 – Clientes y Suscripciones	Registro de clientes y gestión del ciclo de vida de suscripciones a planes de alquiler.	Vendedor, Administrador
M03 – Catálogo de Prendas	Inventario con atributos (talla, color, categoría, estado, valor de reposición) y control de disponibilidad en tiempo real.	Vendedor, Controlador de Stock, Administrador
M04 – Pedidos de Alquiler	Creación, seguimiento y cierre de pedidos. Validación de cupos según plan activo. Restricción: un cliente con paquete activo no puede solicitar otro hasta devolver el anterior.	Vendedor, Administrador
M05 – Despacho	Visualización de pedidos listos para despachar. Verificación de datos. Envío de email al cliente con número de pedido y tracking. Estados: Despachado / En curso / On Hold.	Operador de Inventario, Administrador
M06 – Devoluciones e Incidencias	Inspección de prendas devueltas. Reingreso al catálogo o registro de incidencia (daño, extravío, demora). Cálculo de cargo adicional. Autorización por Supervisor.	Controlador de Stock, Supervisor, Administrador
M07 – Reportes y Auditoría	Consulta de bitácoras (ventas, despachos, indumentaria) con filtros combinados. Medición de rendimiento por vendedor.	Supervisor, Administrador

El alcance del sistema comprende todos los procesos internos relacionados con la

gestión de prendas, clientes, suscripciones, logística de envíos y devoluciones, registro de incidencias, auditoría de operaciones y administración de usuarios y permisos. Quedan explícitamente fuera del alcance la integración con pasarelas de pago externas, la aplicación móvil de autoservicio para el cliente final y cualquier interfaz pública de acceso web.

G03. Definiciones, Acrónimos y Abreviaciones

Definiciones

Término	Definición
CLIENTE	Persona física o jurídica registrada en el sistema. No accede al sistema; interactúa por medios externos (teléfono, WhatsApp, correo).
PRENDA	Unidad de indumentaria del catálogo. Atributos: talla, color, categoría, estado (Disponible / En uso), valor de reposición.
PEDIDO	Solicitud generada por el Vendedor para que un cliente use un conjunto de prendas durante el período de su plan.
SUSCRIPCIÓN	Contrato que define plan, cupo de prendas simultáneas y duración del ciclo.
INCIDENCIA	Evento que afecta el estado de una prenda (daño, extravío, devolución tardía) y puede generar un cargo adicional.
BITÁCORA	Registro cronológico de todas las operaciones realizadas por usuarios del sistema.

Acrónimos y Abreviaciones

Acrónimo	Significado
AES	Advanced Encryption Standard
BE	Business Entities (Capa de Entidades de Negocio)
BLL	Business Logic Layer (Capa de Lógica de Negocio)

CU	Caso de Uso
DAL	Data Access Layer (Capa de Acceso a Datos)
DER	Diagrama de Entidad-Relación
MDI	Multiple Document Interface
POO	Programación Orientada a Objetos
SHA	Secure Hash Algorithm
UI	User Interface (Interfaz de Usuario)

G04. Descripción de Participantes y Roles del Sistema

A continuación se describen los roles previstos para la operación del sistema. Cada rol agrupa responsabilidades funcionales y determina el nivel de acceso a los módulos.

Rol	Responsabilidad	Accesos al Sistema
Administrador	Configuración y gestión general. Alta/baja/modificación de usuarios. Reset de contraseñas.	Todos los módulos. Bitácora de usuarios. Reportes completos.
Vendedor	Primer contacto operativo. Registra clientes, asigna planes, genera pedidos y procesa compras definitivas.	M02, M03 (consulta), M04, compra definitiva, facturación.
Supervisor	Monitoreo y control. Valida incidencias, autoriza cargos, modifica planes (precio y cantidad de prendas).	M06 (autorización de cargos), M07 (reportes y bitácoras de negocio), modificación de planes.

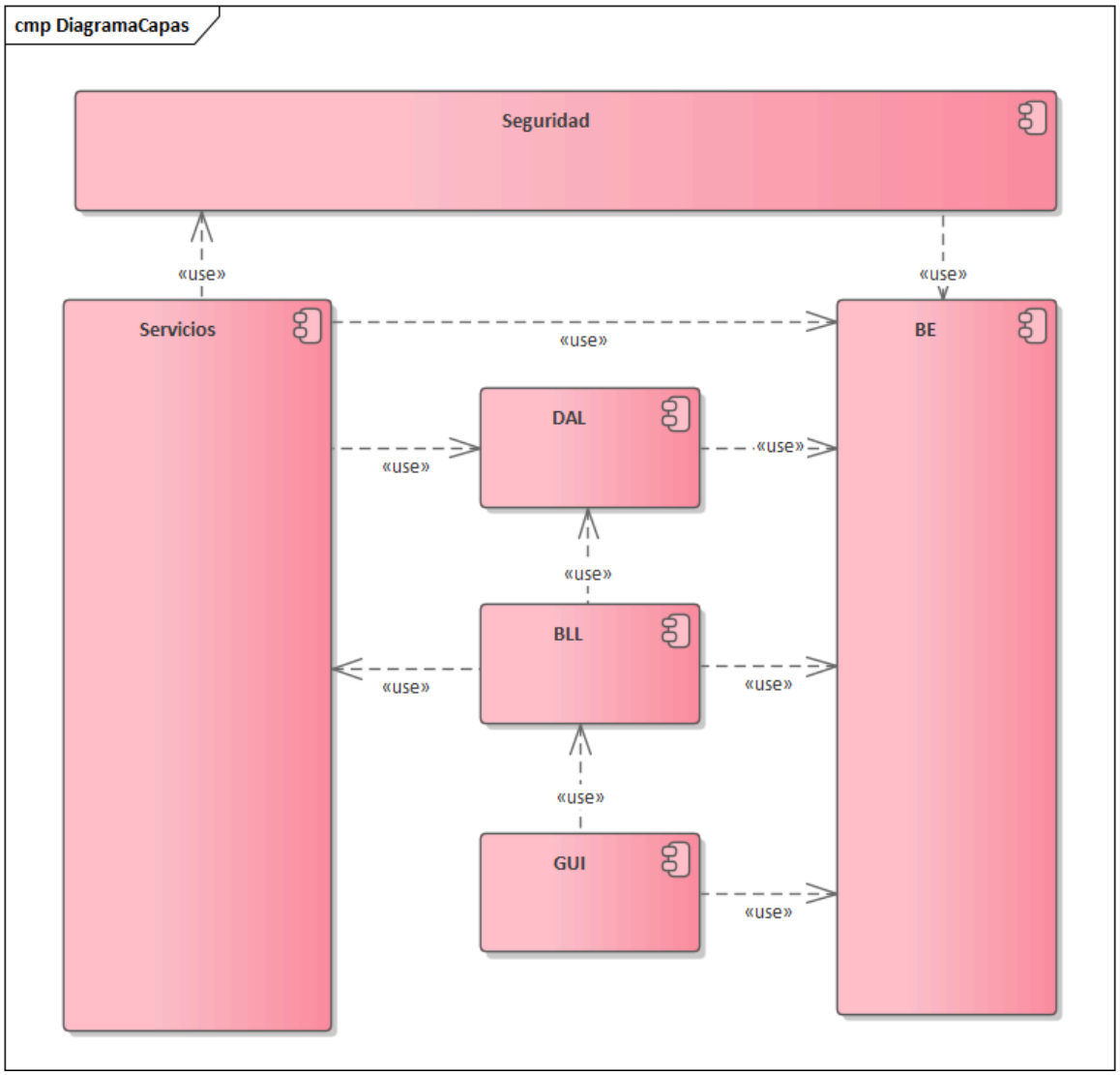
Operador de Inventario	Ejecución de despachos. Vista completamente restringida al módulo de pedidos realizados.	M05 únicamente. Sin acceso a inventario, clientes ni reportes.
Controlador de Stock	Gestión física del catálogo. Inspección de devoluciones, registro de incidencias, mantenimiento de stock.	M03 (gestión), M06 (incidencias). Tablas Prendas y Ventas.
Cliente (externo)	Usuario final del servicio. No accede al sistema en ningún momento.	Sin acceso. Sus acciones son registradas por el Vendedor u Operador.

T01. Arquitectura Base

Objetivo

Definir y documentar la arquitectura base del sistema WardrobeFlow, estableciendo la estructura de capas, el esquema de persistencia, el diagrama de componentes y el mapa de navegación MDI.

Diagrama de Componentes



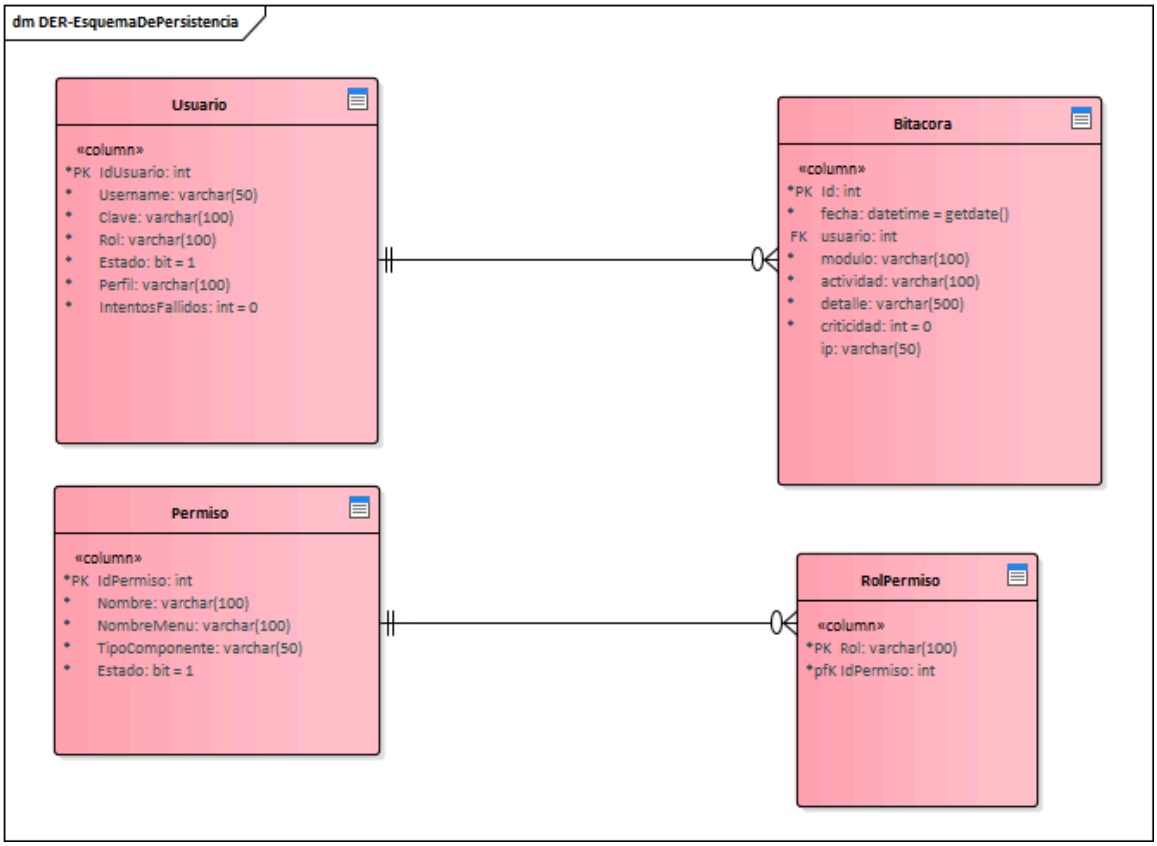
El sistema adopta una arquitectura de capas que separa responsabilidades y minimiza el acoplamiento entre componentes, organizando sus elementos en tres niveles bien definidos tal como se detalla a continuación.

Capa	Responsabilidad	Componentes Principales
Presentación (UI)	Interacción con el usuario mediante formularios MDI. Sin lógica de negocio ni acceso a datos. Delega operaciones a la BLL.	Login, Menu, FormBase, Clientes/ClienteForm, Prendas/PrendaForm, CambioEstadoDialog, Planes, PedidosVenta, PedidosRealizados, NuevoPedidoForm, Bitacora, Usuarios, GestorPermisos, FormIdiomas, VersionHistorialForm + RestauracionForm, DiagnosticoIntegridadForm, BackupForm, dashboards por rol (DashboardForm, DashboardVendedor, DashboardControlStock, DashboardOperador, DashboardSupervisor), OlvideContrasenaForm, ResetClaveDialog, NotificacionDespachoForm, MiPerfilForm
Lógica de Negocio (BLL)	Reglas de dominio, validaciones, orquestación de flujos. Único punto de validación de reglas de negocio.	BLL.Usuario, BLL.Cliente, BLL.Prenda, BLL.Pedido, BLL.PlanSuscripcion, BLL.Configuracion, BLL.Familia, BLL.VersionUsuario, BLL.CuidadorHistorial, BLL.IdiomaService, BLL.Backup, BLL.Bitacora, BLL.ControlMapeado, BLL.Preferencia, BLL.RecuperacionAdmin, BLL.ReporteJornada
Acceso a Datos (DAL)	Comunicación con SQL Server usando ADO.NET. Abstrae operaciones de persistencia.	DAL.Acceso (Singleton), BaseDAL, DAL.Usuario, DAL.Cliente, DAL.Prenda, DAL.Pedido, DAL.Permiso (árbol Composite), DAL.PlanSuscripcion, DAL.Bitacora,

		DAL.BitacoraNegocio, DAL.Empleado, DAL.VersionUsuario (persistencia Memento), DAL.Idioma / DAL.Traduccion / DAL.Control, DAL.DigitoVerificador / DAL.HistorialIntegridad, DAL.ControlMapeado, DAL.ClaveRecuperacion, DAL.Preferencia, DAL.PedidoHistorial, DAL.Backup
Entidades (BE)	Clases del modelo de dominio. Objetos POCO sin lógica de persistencia ni validación.	BE.Usuario (Originator), BE.VersionUsuario + BE.Memento.IMemento / BE.Memento.IOriginator (Memento), BE.Componente / BE.Familia / BE.Patente / BE.Rol + BE.TipoPermiso (Composite), BE.Permiso, BE.Cliente, BE.Prenda, BE.Pedido, BE.PlanSuscripcion, BE.Empleado, BE.Bitacora, BE.BitacoraNegocio, BE.Idioma, BE.FilaUsuarioDV / BE.FilaDV / BE.HistorialIntegridad, BE.AppException / BE.LoginException, enums (BE.Criticidad, BE.EstadoPedido, BE.EstadoPrenda, BE.TipoEventoNegocio), constantes (BE.Roles, BE.Patentes)
Seguridad (transversal)	Autenticación, gestión de sesiones, autorización y encriptación.	SessionManager (Singleton), Encriptador (PBKDF2-SHA256 + AES), CifradorArchivos, DigitoVerificador + CalculadorDV / ICalculadorDV (DVH/DVV), ContadorSesion

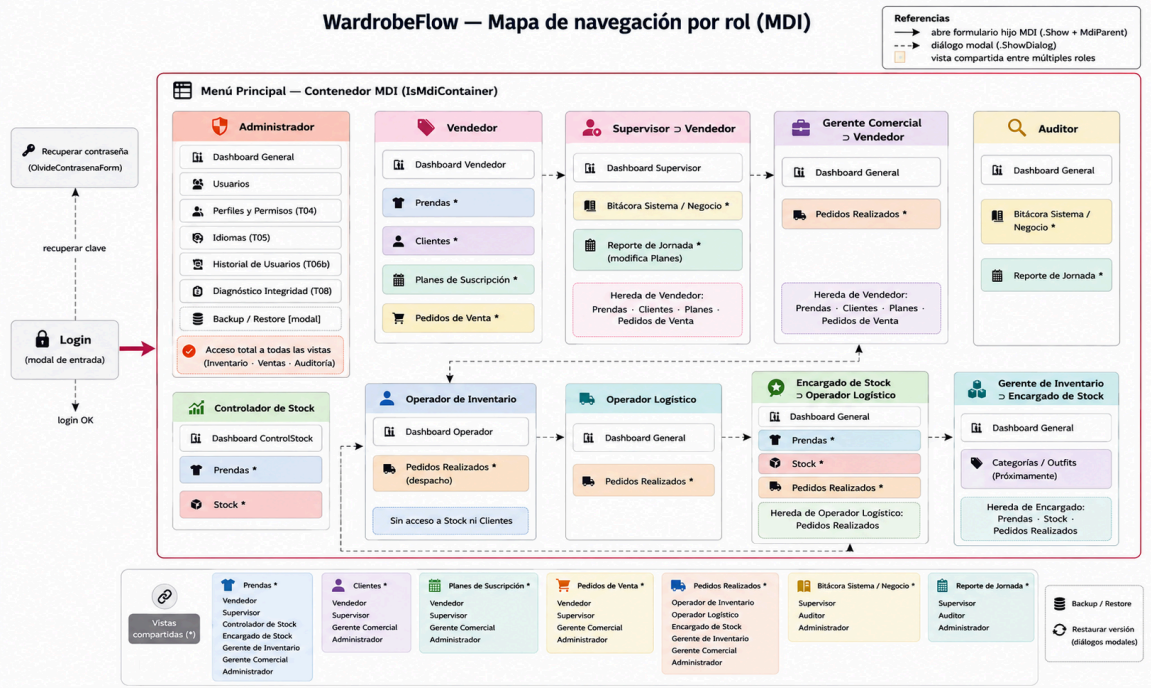
Servicios	Registro de auditoría y eventos de negocio. Capa de orquestación entre BLL y DAL para bitácoras.	Servicios.Bitacora, Servicios.BitacoraNegocio, Servicios.GeneradorCredenciales, Servicios.Multiidioma.GestorIdioma (Observer — Subject), IIdiomaObserver, Traductor / TraductorPerfil / Etiqueta / Traduccion

Esquema de Persistencia



Mapa de Navegación MDI

Los colores representan el rol que accede a cada formulario: violeta (sin sesión), rosa (Administrador), amarillo (Vendedor), verde (ControladorDeStock / OperadorLogistico) y celeste (OperadorLogistico).



Código	Formulario	Descripción	Rol(es)
F00	Login	Pantalla de autenticación. Al iniciarse la aplicación se verifica la conexión con la base de datos; si falla, el sistema informa el	Todos

		error y finaliza la ejecución.	
F00a	OlvideContrasenaForm	Recuperación de contraseña. Solicita username para que un Administrador resetee la clave.	Todos
F01	Menu	Contenedor MDI. Menú dinámico según permisos del usuario autenticado.	Todos
F02	Clientes	ABM de clientes y asignación de plan de suscripción.	Administrador, Vendedor, Supervisor, Gerente Comercial
F02a	ClienteForm	Alta y modificación de clientes.	Administrador, Vendedor, Supervisor, Gerente Comercial
F03	Prendas	Catálogo de prendas con estado en tiempo real y detalle del cliente que la utiliza.	Administrador, Controlador de Stock, Encargado de Stock, Gerente de Inventario, Gerente Comercial
F03a	PrendaForm	Alta y modificación de prendas.	Administrador, Controlador de Stock, Encargado de Stock, Gerente de Inventario

F04	PedidosVenta	Creación y seguimiento de pedidos de alquiler.	Administrador, Vendedor, Supervisor, Gerente Comercial
F04a	NuevoPedidoForm	Creación de un nuevo pedido.	Administrador, Vendedor, Supervisor, Gerente Comercial
F05	PedidosRealizados	Visualización de pedidos listos para despacho.	Administrador, Operador Logístico, Operador de Inventario, Encargado de Stock, Gerente de Inventario, Gerente Comercial
F05a	NotificacionDespacho Form	Notificación de despacho de pedidos.	Administrador, Operador Logístico, Operador de Inventario, Encargado de Stock, Gerente de Inventario
F06	Planes	Gestión de planes de suscripción.	Administrador, Vendedor, Supervisor, Gerente Comercial
F07	Bitacora	Consulta de bitácora del sistema y de negocio.	Administrador, Auditor, Supervisor

F08	Usuarios	Gestión de usuarios, asignación de roles, desbloqueo y reseteo de contraseñas.	Administrador

T02. Gestión de Login / Logout del Sistema

T02.1 Gestión de Login del Sistema

Objetivo

Verificar la identidad del usuario mediante credenciales, asignarle el perfil correspondiente y habilitar acceso exclusivamente a las funcionalidades autorizadas por su rol.

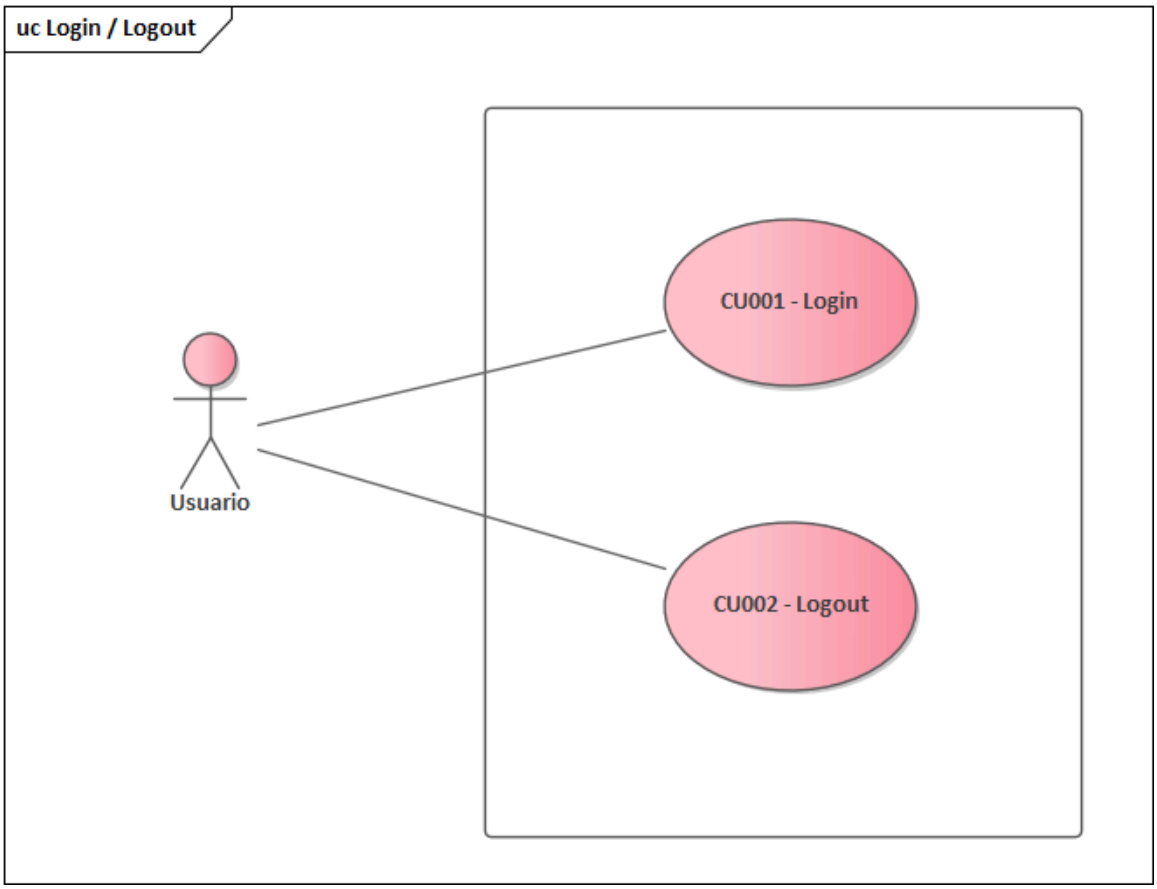
Descripción detallada de funcionamiento

El sistema proporciona un mecanismo de autenticación que permite a los usuarios acceder al entorno del sistema mediante el ingreso de su nombre de usuario y contraseña. Al iniciar la aplicación, el sistema verifica que la conexión con la base de datos sea exitosa. Si la conexión falla, se informa el error al usuario y la aplicación finaliza su ejecución sin mostrar el formulario de login.

Una vez autenticado, el usuario accede únicamente a las funcionalidades habilitadas según su perfil, tales como la gestión de prendas, administración de cuentas, configuración del sistema, visualización de reportes y estadísticas operativas.

Para reforzar la seguridad, el sistema controla los intentos de acceso fallidos: si un usuario ingresa credenciales incorrectas más de tres veces consecutivas, su cuenta se bloquea automáticamente, impidiendo nuevos intentos hasta que un Administrador la rehabilite.

Diagrama de Casos de Uso



Especificación de Caso de Uso

ID Y NOMBRE:	CU 001 - Login
DESCRIPCIÓN: El usuario desea acceder al sistema utilizando sus credenciales de inicio de sesión para poder usar las funcionalidades correspondientes a su rol.	
ACTOR PRINCIPAL: Usuario del Sistema	

ACTORES SECUNDARIO: -

PRECONDICIONES:

- La aplicación debe estar correctamente instalada y ejecutándose.
- El usuario debe contar con una cuenta registrada y activa en el sistema.

PUNTOS DE EXTENSIÓN:

PUNTOS DE INCLUSIÓN:

ESCENARIO PRINCIPAL:

1. El usuario accede a la pantalla de login e ingresa su nombre de usuario y contraseña.
2. El sistema busca el usuario en la base de datos, verifica que la cuenta no esté bloqueada y valida las credenciales contra el hash almacenado.
3. El sistema carga los permisos asociados al rol del usuario, otorgándole acceso únicamente a las funciones habilitadas.
4. El sistema muestra el menú principal con las opciones disponibles según el perfil del usuario y registra el evento de inicio de sesión en la bitácora.

FLUJOS ALTERNATIVOS:

2.1 Si las credenciales son incorrectas o inválidas, el sistema muestra un mensaje de error y solicita al usuario que las ingrese nuevamente.

2.2 Si la cuenta del usuario se encuentra bloqueada, el sistema muestra un mensaje advirtiendo la situación y sugiere contactar al administrador.

2.3 Si se superan los 3 intentos fallidos consecutivos de acceso, la cuenta se bloquea automáticamente y se muestra el mensaje correspondiente.

POSTCONDICIONES:

- Si la autenticación fue exitosa, el usuario accede a la pantalla principal del sistema.

- Si el acceso no fue validado, el usuario permanece en la pantalla de login sin avanzar.

Diagrama de Clases

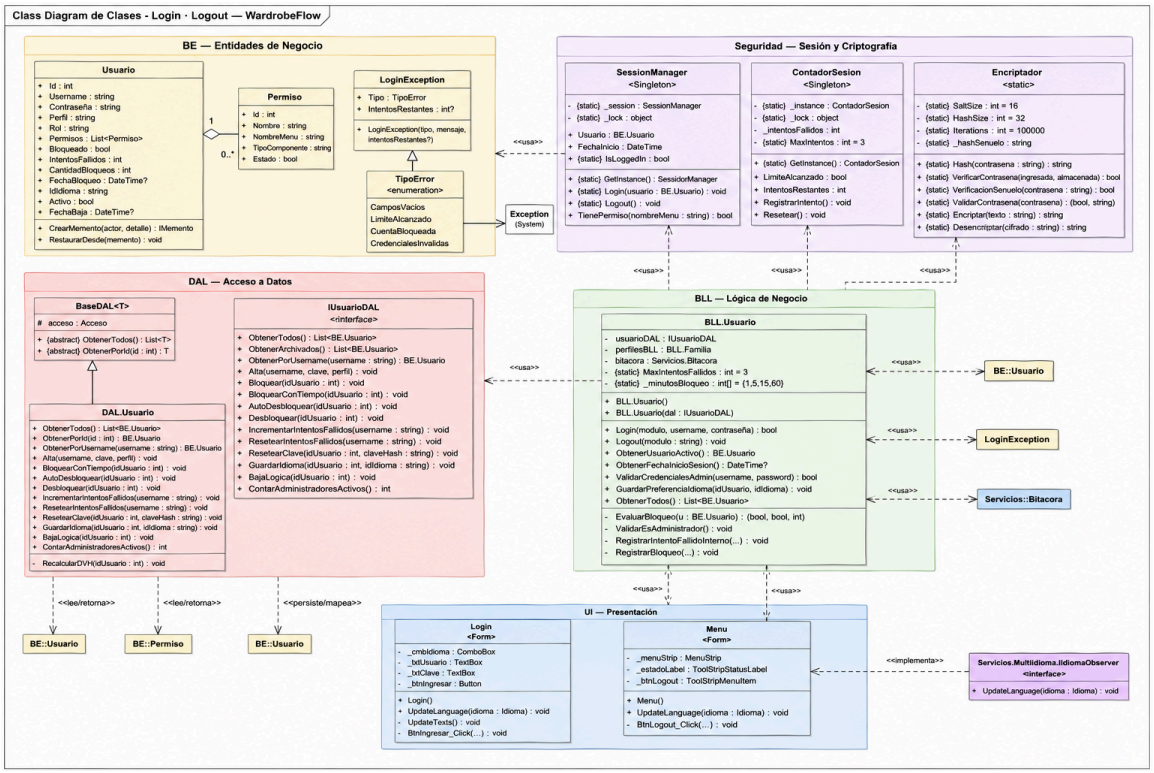
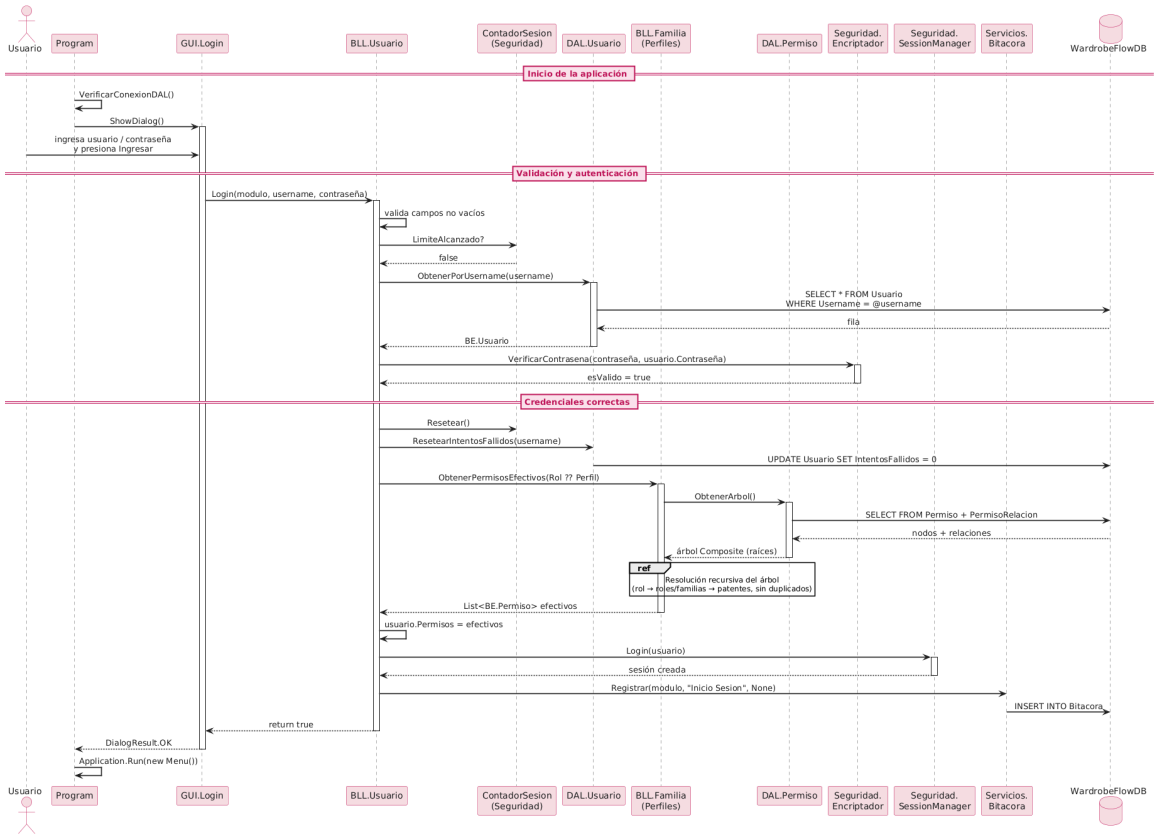
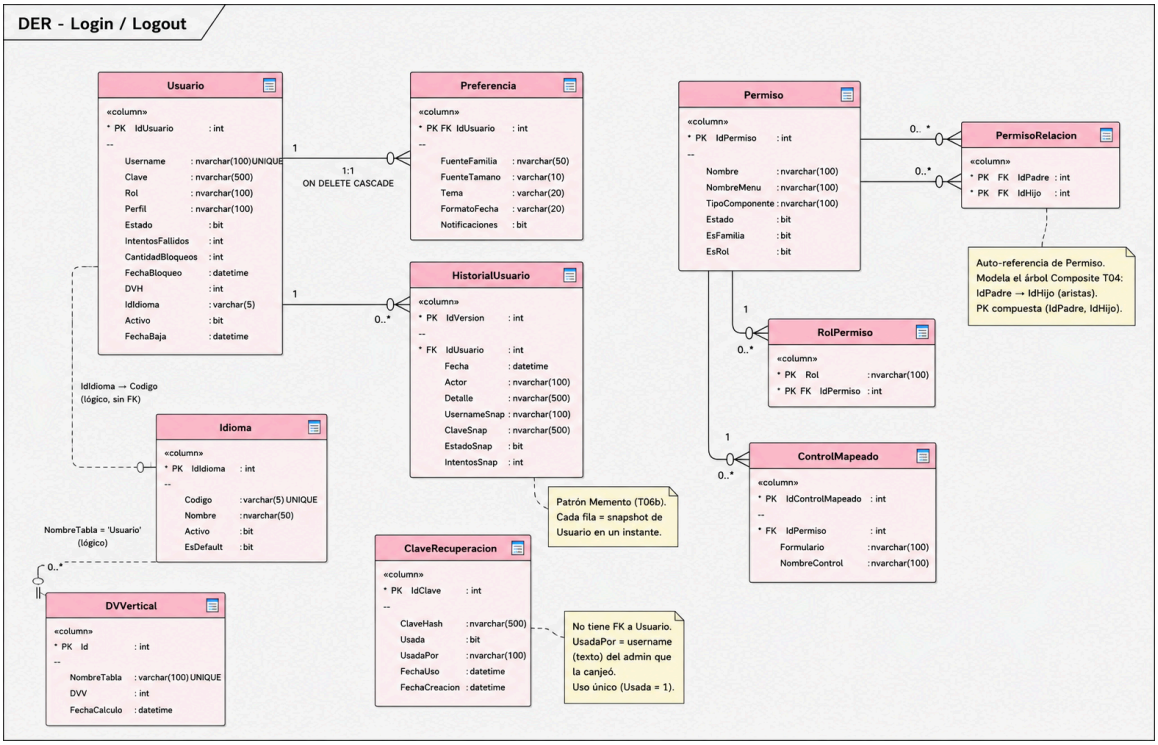


Diagrama de Secuencia



DER


DER - Login / Logout
Usuario
«column»
PK IdUsuario : int
Username : nvarchar(100) UNIQUE
Clave : nvarchar(500)
Rol : nvarchar(100)
Perfil : nvarchar(100)
Estado : bit
IntentosFallidos : int
CantidadBloqueos : int
FechaBloqueo : datetime
DVH : int
IdIdioma : varchar(5)
Activo : bit
FechaBaja : datetime
1
1:1
ON DELETE CASCADE
Preferencia
«column»
PK FK IdUsuario : int
FuenteFamilia : nvarchar(50)
FuenteTamano : varchar(10)
Tema : varchar(20)
FormatoFecha : varchar(20)
Notificaciones : bit
1
0..*
HistorialUsuario
«column»
PK IdVersion : int
FK IdUsuario : int
Fecha : datetime
Actor : nvarchar(100)
Detalle : nvarchar(500)
UsernameSnap : nvarchar(100)
ClaveSnap : nvarchar(500)
EstadoSnap : bit
IntentosSnap : int
Patrón Memento (T06b).
Cada fila = snapshot de
Usuario en un instante.
IdIdioma → Codigo
(lógico, sin FK)
Idioma
«column»
PK IdIdioma : int
Codigo : varchar(5) UNIQUE
Nombre : nvarchar(50)
Activo : bit
EsDefault : bit
NombreTabla = 'Usuario'
(lógico)
0..*
DVVertical
«column»
PK Id : int
NombreTabla : varchar(100) UNIQUE
DVV : int
FechaCalculo : datetime
ClaveRecuperacion
«column»
PK IdClave : int
ClaveHash : nvarchar(500)
Usada : bit
UsadaPor : nvarchar(100)
FechaUso : datetime
FechaCreacion : datetime
No tiene FK a Usuario.
UsadaPor = username
(texto) del admin que
la canjeó.
Uso único (Usada = 1).
Permiso
«column»
PK IdPermiso : int
Nombre : nvarchar(100)
NombreMenu : nvarchar(100)
TipoComponente : nvarchar(100)
Estado : bit
EsFamilia : bit
EsRol : bit
0..*
0..*
PermisoRelacion
«column»
PK FK IdPadre : int
PK FK IdHijo : int
Auto-referencia de Permiso.
Modela el árbol Composite T04:
IdPadre → IdHijo (aristas).
PK compuesta (IdPadre, IdHijo).
1
0..*
RolPermiso
«column»
PK Rol : nvarchar(100)
PK FK IdPermiso : int
1
0..*
ControlMapeado
«column»
PK IdControlMapeado : int
FK IdPermiso : int
Formulario : nvarchar(100)
NombreControl : nvarchar(100)

T02.2 Gestión de Logout del Sistema

Objetivo

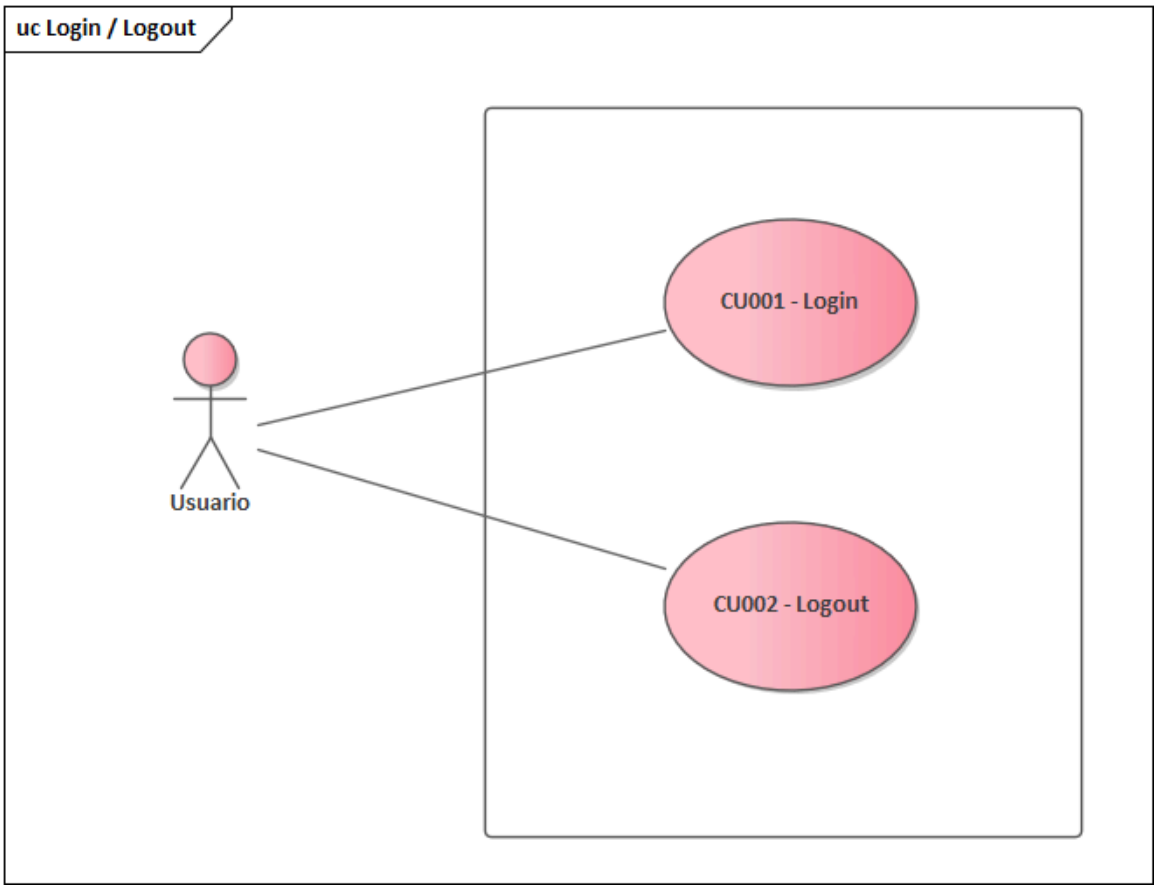
Permitir al usuario finalizar su sesión de forma segura, liberando el contexto de seguridad y garantizando que ningún otro usuario acceda sin autenticarse nuevamente.

Descripción detallada de funcionamiento

El sistema ofrece una funcionalidad de cierre de sesión que permite al usuario finalizar su sesión de forma segura desde el menú principal. Al seleccionar la opción, el sistema solicita confirmación mediante un diálogo. Si el usuario confirma, se registra el evento en la bitácora, se destruye la instancia Singleton de sesión y el sistema reinicia la aplicación, cerrando todos los formularios abiertos y redirigiendo al usuario a la pantalla de login para una nueva autenticación.

Una vez cerrada la sesión, ningún módulo del sistema puede acceder a información protegida sin que se realice una nueva autenticación, garantizando así la integridad y confidencialidad de los datos.

Diagrama de Caso de Uso



Especificación de Caso de Uso

ID Y NOMBRE:	CU 002 - Logout
DESCRIPCIÓN: El usuario desea cerrar su sesión activa en el sistema de forma segura, liberando el contexto de seguridad y garantizando que ningún otro usuario pueda acceder a sus funcionalidades sin autenticarse nuevamente.	

ACTOR PRINCIPAL: Usuario del Sistema

ACTORES SECUNDARIO: -

PRECONDICIONES:

- La aplicación debe estar correctamente instalada y ejecutándose.
- El usuario debe tener una sesión activa en el sistema.

PUNTOS DE EXTENSIÓN: -

PUNTOS DE INCLUSIÓN: -

ESCENARIO PRINCIPAL:

1. El usuario selecciona Perfil → Cerrar Sesión desde el menú principal y el sistema solicita confirmación mediante un diálogo.
2. El usuario confirma el cierre de sesión.
3. El sistema registra el evento de cierre en la bitácora y elimina los datos de sesión del usuario activo, destruyendo la instancia Singleton.
4. El sistema reinicia la aplicación, cierra todos los formularios abiertos y redirige al usuario a la pantalla de login.

FLUJOS ALTERNATIVOS:

1.1 Si el usuario no confirma el cierre de sesión, el sistema cancela la operación y el usuario permanece en la pantalla actual sin modificaciones.

POSTCONDICIONES:

- Si el cierre de sesión fue exitoso, el sistema regresa a la pantalla de login.
- Si el usuario no confirma el cierre, permanece en la pantalla actual sin modificaciones.

Universidad Abierta Interamericana	**UNIVERSIDAD ABIERTA INTERAMERICANA** **Facultad de Tecnología Informática**			
	Materia: Ingeniería de Software		**Docente**: Juan Ignacio Silva	**Fecha** 30/04
	Alumnas: Bolivar, Magali - Morana, Valentina			
	Sede: Centro	**Comisión**: 3A	**Turno**: Noche	**Año**: 2026
Sistema de Gestión de Alquiler de Prendas por Suscripción (Compra-Venta)				**Versión** 1.0.0

Diagrama de Clases

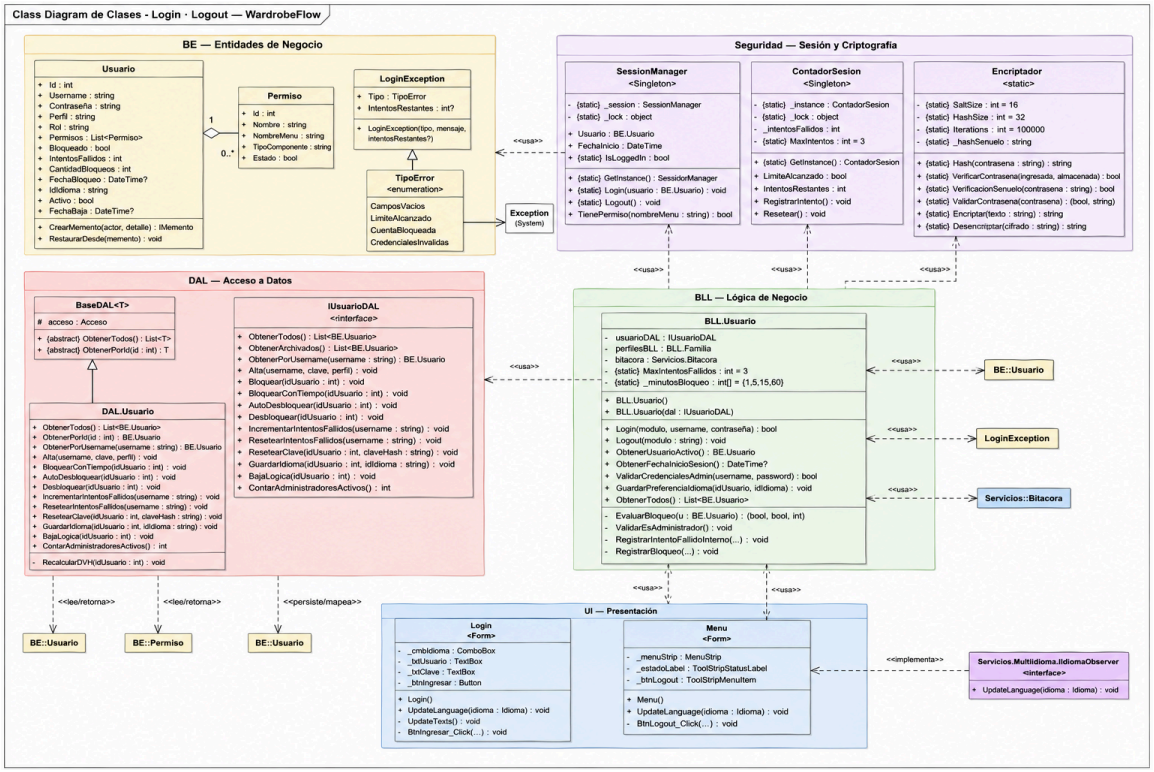
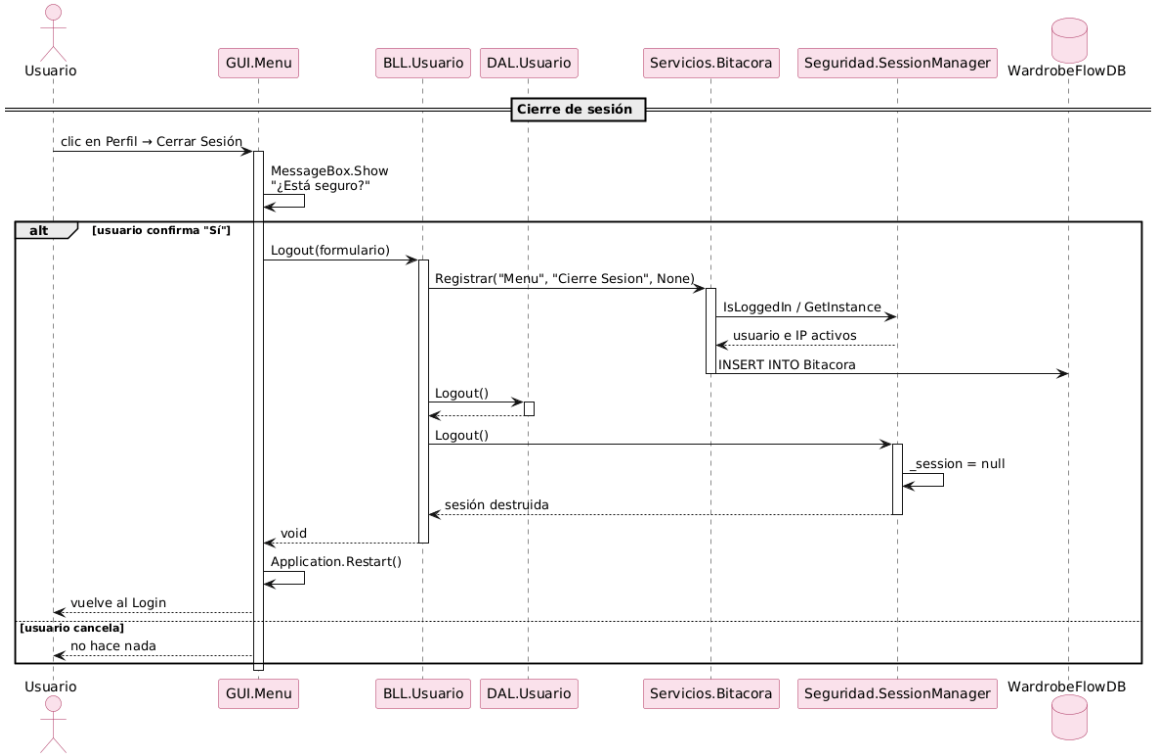
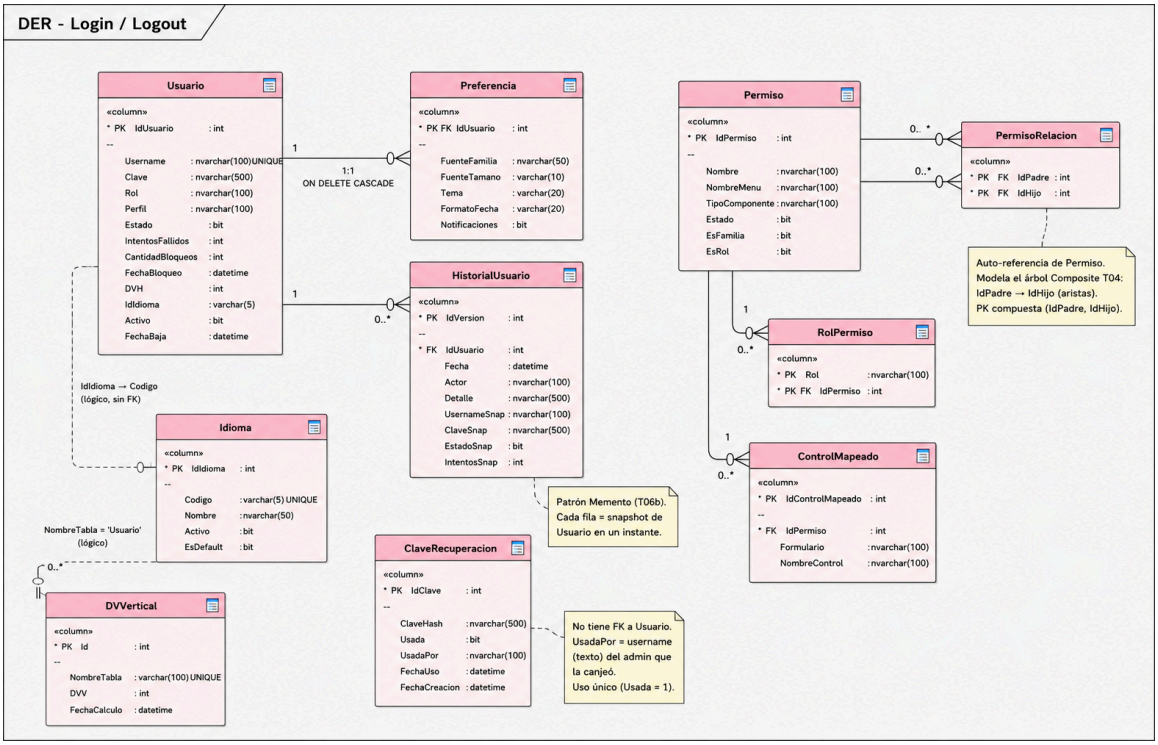


Diagrama de Secuencia


Usuario
GUI.Menu
BLL.Usuario
DAL.Usuario
Servicios.Bitacora
Seguridad.SessionManager
WardrobeFlowDB
Cierre de sesión
clic en Perfil → Cerrar Sesión
MessageBox.Show
"¿Está seguro?"
alt
[usuario confirma "Sí"]
Logout(formulario)
Registrar("Menu", "Cierre Sesion", None)
IsLoggedIn / GetInstance
usuario e IP activos
INSERT INTO Bitacora
Logout()
Logout()
_session = null
sesión destruida
void
Application.Restart()
vuelve al Login
[usuario cancela]
no hace nada
Usuario
GUI.Menu
BLL.Usuario
DAL.Usuario
Servicios.Bitacora
Seguridad.SessionManager
WardrobeFlowDB

DER

Universidad Abierta Interamericana	**UNIVERSIDAD ABIERTA INTERAMERICANA** **Facultad de Tecnología Informática**			
	Materia: Ingeniería de Software		**Docente**: Juan Ignacio Silva	**Fecha** 30/04
	Alumnas: Bolivar, Magali - Morana, Valentina			
	Sede: Centro	**Comisión**: 3A	**Turno**: Noche	**Año**: 2026
Sistema de Gestión de Alquiler de Prendas por Suscripción (Compra-Venta)				**Versión** 1.0.0



T03. Gestión de Encriptado

Objetivo

Proteger los datos sensibles del sistema mediante algoritmos criptográficos robustos. Se implementa un mecanismo de hash irreversible con salt para contraseñas (PBKDF2-SHA256), resistente a ataques de fuerza bruta, tablas rainbow y diccionario. Adicionalmente, se implementa cifrado simétrico AES-128-CBC para proteger datos sensibles almacenados en la base de datos.

Toda la lógica criptográfica reside en la clase estática Encriptador (capa Seguridad), cumpliendo el principio Open/Closed de SOLID: cambiar un algoritmo no impacta a los módulos consumidores (BLL.Usuario, DAL.Cliente).

Descripción detallada de funcionamiento

La gestión de encriptado es el mecanismo del sistema responsable de proteger la información sensible mediante transformaciones criptográficas. Se implementa a través de la clase Encriptador, que centraliza dos algoritmos distintos. Esta separación responde al principio Open/Closed de SOLID: toda la lógica criptográfica reside en un único componente, de modo que cualquier modificación algorítmica futura no impacta a los módulos que lo consumen, como BLL.Usuario o DAL.Cliente.

Algoritmos Implementados – Hash de Contraseñas PBKDF2-SHA256

Se utiliza PBKDF2 implementado mediante la clase Rfc2898DeriveBytes de .NET con HMAC-SHA256.

Parámetro	Valor	Justificación
Algoritmo base	HMAC-SHA256	Estándar ampliamente auditado y recomendado por NIST
Salt	16 bytes aleatorios (RandomNumberGenerator)	Previene ataques de tablas rainbow y diccionario precomputados

Iteraciones	100.000	Costo computacional suficiente para resistir fuerza bruta en hardware moderno
Longitud del hash	32 bytes (256 bits)	256 bits de seguridad efectiva
Formato almacenado	Base64(Salt[16] + Hash[32]) = 64 chars	Campo único NCHAR(64) en tabla Usuario (Clave)

Algoritmos Implementados – Cifrado Simétrico AES-128-CBC

Se utiliza AES-128 en modo CBC con IV aleatorio por operación. Al ser reversible, permite recuperar el valor original cuando el acceso está autorizado.

Parámetro	Valor	Justificación
Algoritmo	AES-128 (CBC)	Estándar simétrico ampliamente auditado
Longitud de clave	16 bytes (128 bits), generada por GenerarClave()	Clave derivada de semilla fija (ambiente académico)
IV (Vector de Inicialización)	16 bytes aleatorios por operación	Garantiza que dos cifrados del mismo valor produzcan resultados distintos
Formato almacenado	Base64(IV[16] + CipherText) → NVARCHAR en BD	Texto legible para almacenamiento en columnas de texto
Datos protegidos	DNI del cliente (DAL.Cliente)	Dato de identidad sensible. Se muestra descifrado en pantalla

Flujo de aplicación por caso de uso

Caso de uso	Método	Detalle

Login de usuario	VerificarContraseña(ingresada, almacenada)	Extrae salt, rehashea y compara byte a byte
Registro / cambio de contraseña	Hash(contraseña)	Almacena solo el hash, nunca texto plano
Guardar DNI del cliente	Encriptar(nro_doc)	Cifra con AES-128-CBC + IV aleatorio. Almacena Base64
Leer DNI del cliente	Desencriptar(valor_bd)	Extrae IV, aplica AES inverso y retorna texto claro
Leer DNI (registro previo sin cifrar)	TryDesencriptar(valor_bd)	Intenta descifrar; si falla, retorna el valor original sin modificar

Descripción de métodos principales

Método	Tipo	Descripción
Hash(texto): string	Unidireccional	PBKDF2-SHA256. Retorna 64 chars Base64. Para contraseñas
VerificarContraseña (ingresada, almacenada): bool	Unidireccional	Extrae salt del hash almacenado, rehashea y compara byte a byte
Encriptar(texto): string	AES-128	Cifra con AES-128-CBC + IV aleatorio. Retorna Base64
Desencriptar(cifrado): string	AES-128	Decodifica Base64, extrae IV y aplica AES inverso
TryDesencriptar(valor): string	AES-128	Igual que Desencriptar, pero ante error retorna el valor original

GenerarClave(): byte[]	Auxiliar	Genera clave AES de 16 bytes desde semilla fija (ambiente académico)
ConvertirBase64(data): string	Auxiliar	Convierte bytes a Base64 para almacenamiento en BD

T04. Gestión de Perfiles de Usuario

Objetivo

Implementar un motor de autorización dinámico, jerárquico y flexible que permita agrupar permisos en familias y roles, facilitando la administración de accesos y garantizando que un usuario solo pueda ejecutar las acciones para las que está autorizado.

Descripción funcional

Los permisos del sistema se organizan de forma jerárquica utilizando una estructura de árbol. Para modelar esta estructura se implementó el patrón Composite, permitiendo administrar permisos individuales y grupos de permisos de manera uniforme.

La solución está compuesta por las siguientes clases:

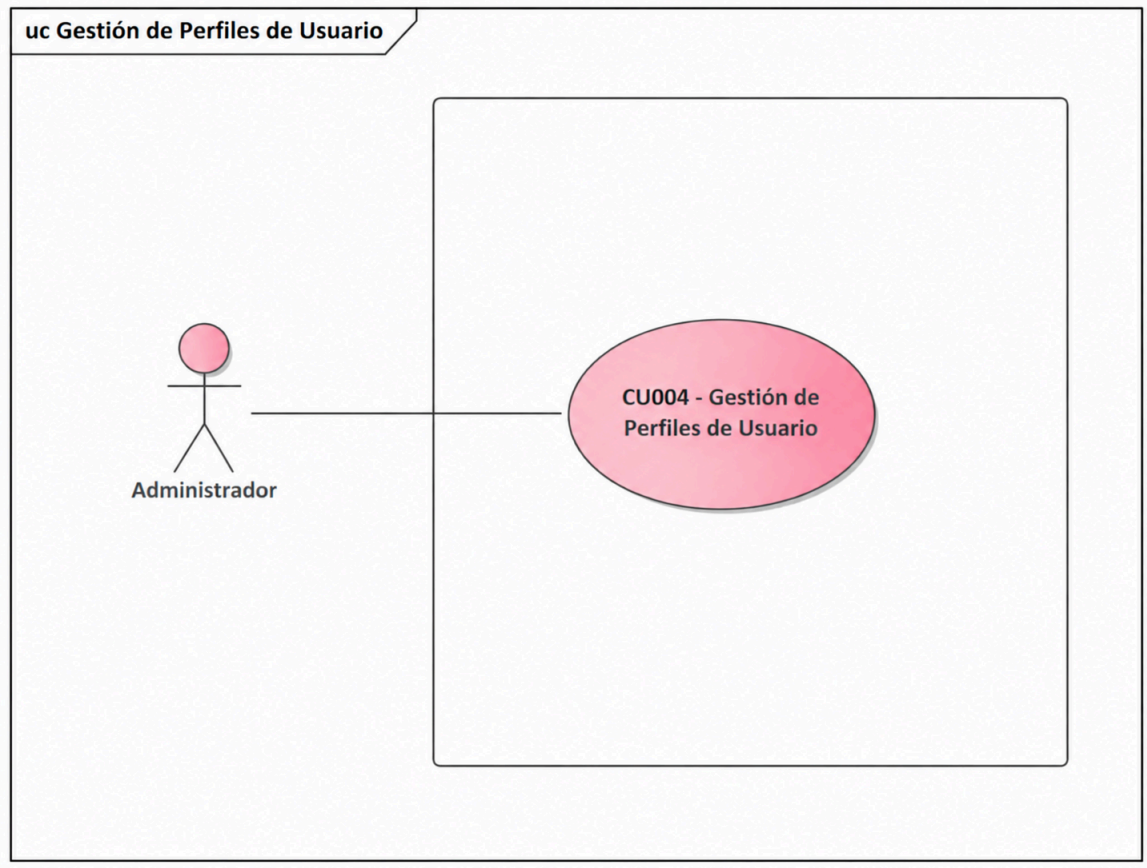
- Componente: define las operaciones comunes para todos los elementos de la estructura.
- Familia: agrupa permisos y otros elementos relacionados.
- Patente: representa un permiso individual.
- Rol: hereda de Familia y representa un perfil asignable a los usuarios.

Gracias a esta estructura, un rol puede contener permisos individuales, familias completas e incluso otros roles, facilitando la reutilización y administración de permisos.

Para determinar los permisos disponibles de un usuario, el sistema recorre la estructura y obtiene todas las patentes asociadas al rol asignado.

La información se almacena en la base de datos mediante las tablas Permiso y PermisoRelacion, desde donde se reconstruye el árbol de permisos al momento de su utilización.

Caso de Uso



Especificación de Caso de Uso

ID Y NOMBRE:	CU 004 - Gestión de Perfiles de Usuario
DESCRIPCIÓN: Permite al Administrador gestionar la estructura de permisos del sistema. Puede crear, modificar y eliminar patentes (permisos atómicos), familias (grupos de	

permisos) y roles (perfiles asignables a usuarios), y configurar qué componentes pertenecen a cada contenedor. Los permisos efectivos de un nodo se resuelven de forma recursiva y se actualizan en vivo.

ACTOR PRINCIPAL: Administrador

ACTORES SECUNDARIO: -

PRECONDICIONES:

- El usuario debe tener una sesión activa con permiso de gestión de usuarios.
- Debe existir al menos un componente (patente, familia o rol) en el árbol de permisos.

PUNTOS DE EXTENSIÓN: -

PUNTOS DE INCLUSIÓN: -

ESCENARIO PRINCIPAL:

1. El usuario accede a Administrar → Gestión de Perfiles desde el menú principal.
2. El sistema carga y muestra el árbol de permisos completo (roles, familias y patentes), junto con los paneles de componentes disponibles y miembros del nodo seleccionado.
3. El usuario selecciona un nodo del árbol (rol o familia).
4. El sistema muestra los componentes disponibles para agregar, los miembros actuales del nodo y los permisos efectivos calculados de forma recursiva.
5. El usuario realiza la operación deseada (crear, agregar, quitar, renombrar o eliminar).
6. El sistema valida y persiste el cambio en la base de datos.
7. El sistema registra la operación en la bitácora, recarga el árbol y actualiza los permisos en vivo.

FLUJOS ALTERNATIVOS:

5.1 El usuario crea una patente, familia o rol: el sistema solicita el nombre, valida que no esté vacío y que el rol no esté duplicado, crea el componente y continúa en el paso 6.

5.2 El usuario agrega un componente a un contenedor: el sistema verifica que la operación no genere una referencia circular en el árbol. Si genera ciclo, muestra un error y cancela la operación.

5.3 El usuario quita un componente de un contenedor: el sistema elimina la relación sin borrar el componente del sistema.

5.4 El usuario renombra un componente: el sistema solicita el nuevo nombre, valida que no esté vacío y actualiza el registro.

5.5 El usuario elimina un componente: el sistema solicita confirmación. Si el componente es un rol con usuarios asignados, muestra los nombres afectados y cancela la operación. Si no tiene usuarios asignados, realiza la baja lógica y elimina sus relaciones.

5.6 El usuario intenta quitarse a sí mismo el permiso de gestión sobre su propio rol: el sistema detecta el autobloqueo potencial y cancela la operación con un mensaje explicativo.

POSTCONDICIONES:

- La estructura de permisos queda actualizada en la base de datos.
- Los cambios se reflejan en el árbol visible y en el menú principal del usuario en sesión.
- Todas las operaciones quedan registradas en la bitácora del sistema.

Diagrama de clases

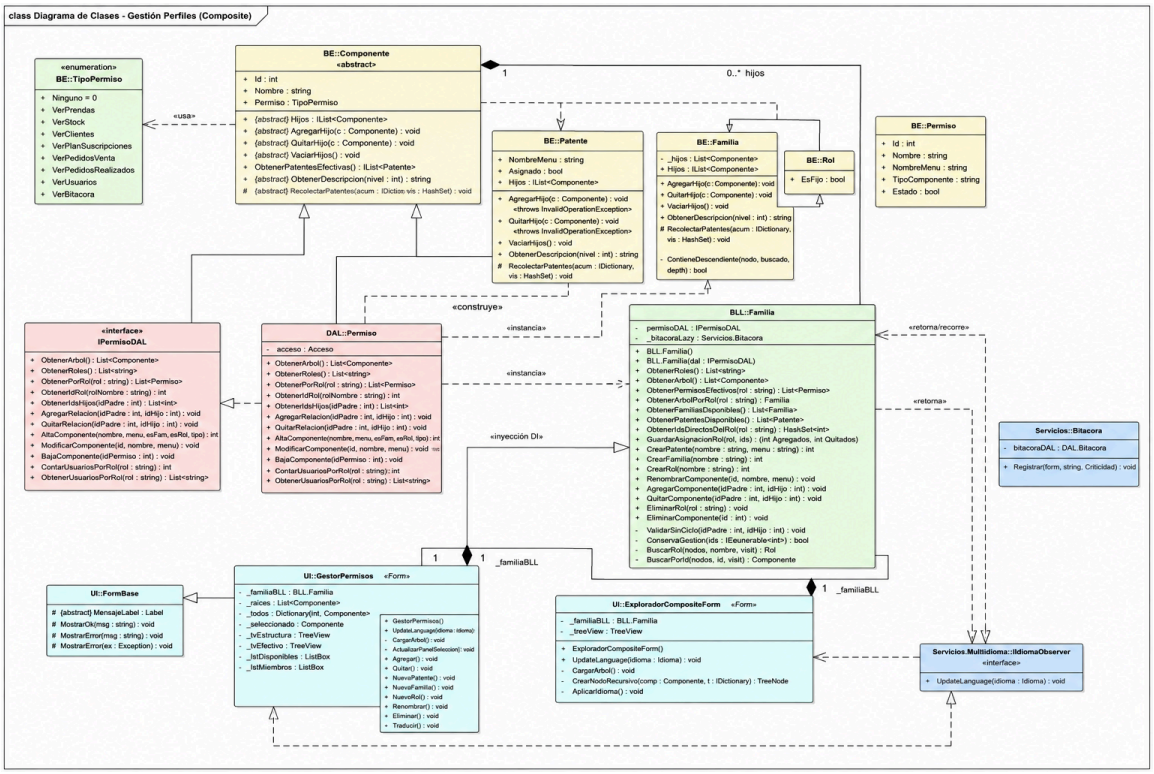
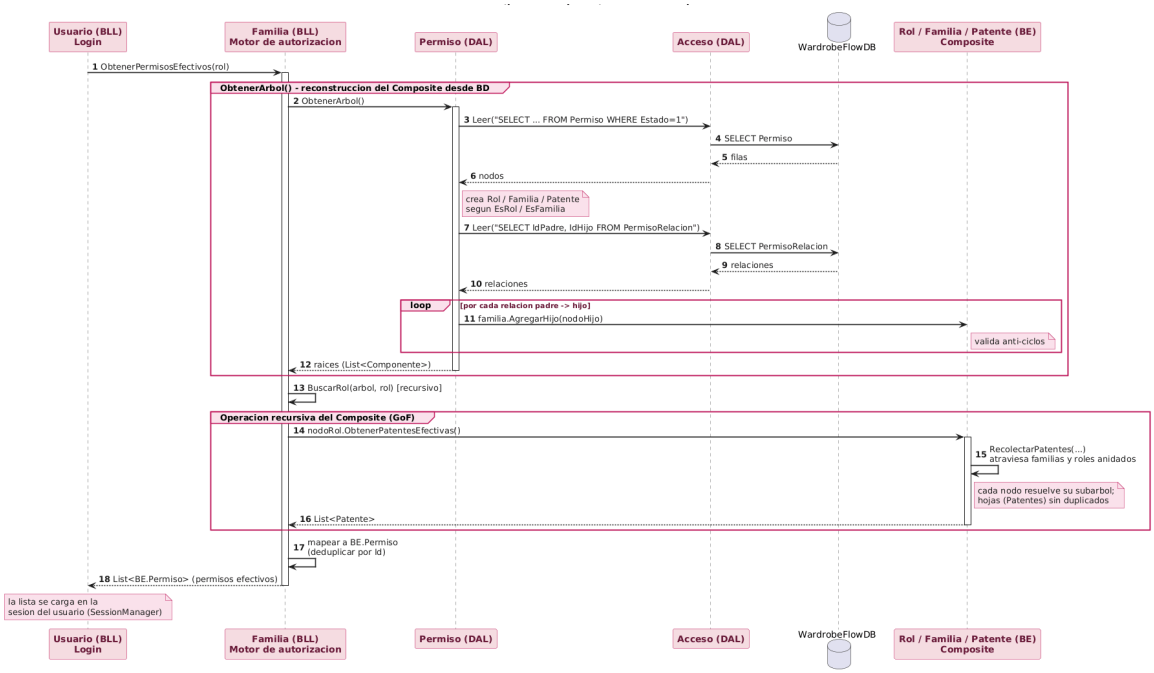
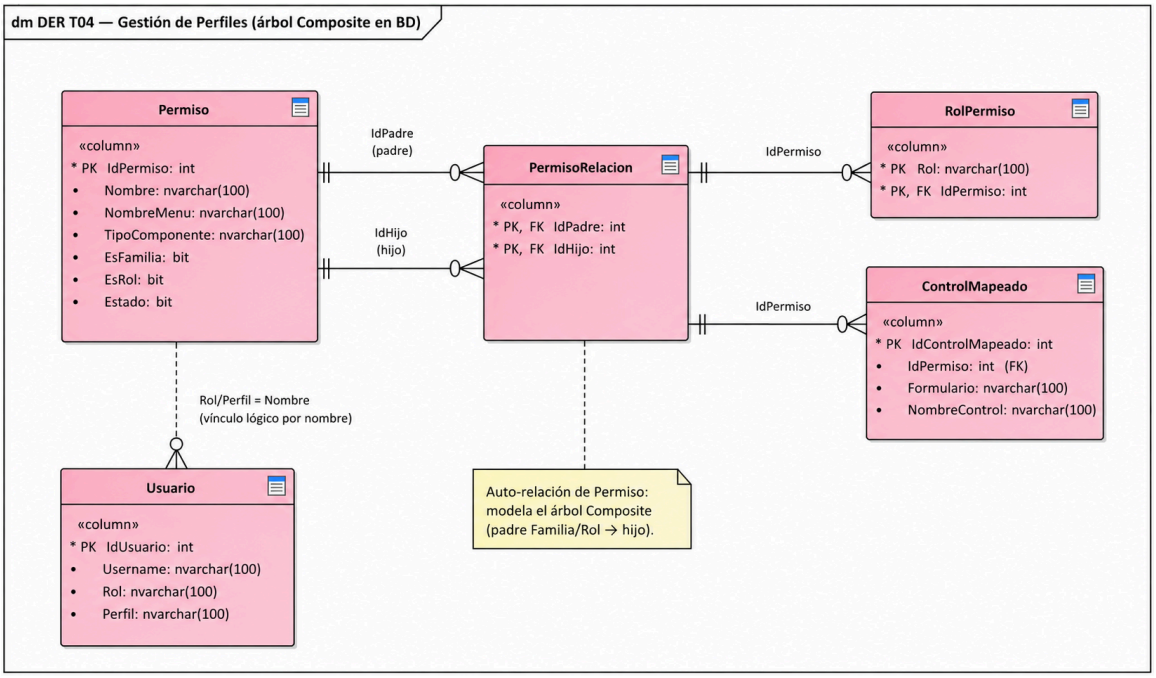


Diagrama de secuencia


Usuario (BLL) Login
Familia (BLL) Motor de autorizacion
Permiso (DAL)
Acceso (DAL)
WardrobeFlowDB
Rol / Familia / Patente (BE) Composite
1 ObtenerPermisosEfectivos(rol)
ObtenerArbol() - reconstruccion del Composite desde BD
2 ObtenerArbol()
3 Leer("SELECT ... FROM Permiso WHERE Estado=1")
4 SELECT Permiso
5 filas
6 nodos
crea Rol / Familia / Patente segun EsRol / EsFamilia
7 Leer("SELECT IdPadre, IdHijo FROM PermisoRelacion")
8 SELECT PermisoRelacion
9 relaciones
10 relaciones
loop
[por cada relacion padre -> hijo]
11 familia.AgregarHijo(nodoHijo)
valida anti-ciclos
12 raices (List<Componente>)
13 BuscarRol(arbol, rol) [recursivo]
Operacion recursiva del Composite (GoF)
14 nodoRol.ObtenerPatentesEfectivas()
15 RecolectarPatentes(...) atraviesa familias y roles anidados
cada nodo resuelve su subarbol; hojas (Patentes) sin duplicados
16 List<Patente>
17 mapear a BE.Permiso (deduplicar por Id)
18 List<BE.Permiso> (permisos efectivos)
la lista se carga en la sesion del usuario (SessionManager)

DER



T05. Gestión de Múltiples Idiomas

Objetivo

Permitir que la aplicación funcione en distintos idiomas y que los usuarios puedan cambiar el idioma de la interfaz en tiempo de ejecución, sin necesidad de reiniciar la aplicación. Además, el sistema debe permitir la incorporación de nuevos idiomas desde la propia administración y actualizar automáticamente todas las ventanas abiertas cuando se produzca un cambio.

Descripción funcional

La solución de multiidioma se implementa mediante el patrón Observer (GoF), permitiendo que todos los formularios de la aplicación sean notificados automáticamente cuando el usuario selecciona un nuevo idioma.

Actualmente el sistema incluye soporte para los idiomas Español (ES), Inglés (EN), Ruso (RU) y Portugués (PT), aunque la arquitectura permite incorporar nuevos idiomas sin necesidad de realizar modificaciones en el código fuente.

Estructura de componentes

La implementación está compuesta por los siguientes elementos:

- **IIdiomaObserver:** interfaz implementada por los formularios que deben actualizar sus textos cuando cambia el idioma.
- **GestorIdioma:** componente responsable de administrar el idioma activo y notificar los cambios a los formularios registrados.
- **Traductor:** encargado de obtener y proporcionar las traducciones disponibles.

Funcionamiento

Cuando un formulario se abre, se registra en el gestor de idiomas para recibir notificaciones. De esta manera, cada vez que el usuario selecciona un idioma diferente, el sistema informa automáticamente el cambio a todos los formularios abiertos, permitiendo que actualicen sus textos de forma inmediata.

Actualmente, diecinueve formularios de la aplicación participan de este mecanismo, garantizando una experiencia consistente en toda la interfaz.

Gestión de traducciones

Las traducciones se almacenan principalmente en la base de datos, desde donde son cargadas y mantenidas en memoria para optimizar el rendimiento de la aplicación.

Como mecanismo de respaldo, el sistema dispone de un conjunto inicial de traducciones definidas en código. Esto permite que la aplicación continúe funcionando correctamente incluso si la base de datos no se encuentra disponible o aún no contiene información de idiomas.

Durante el primer inicio, si no existen traducciones registradas, el sistema genera automáticamente la información inicial a partir de estos datos de respaldo.

Administración de idiomas

La gestión de idiomas se realiza mediante un formulario específico accesible para los administradores.

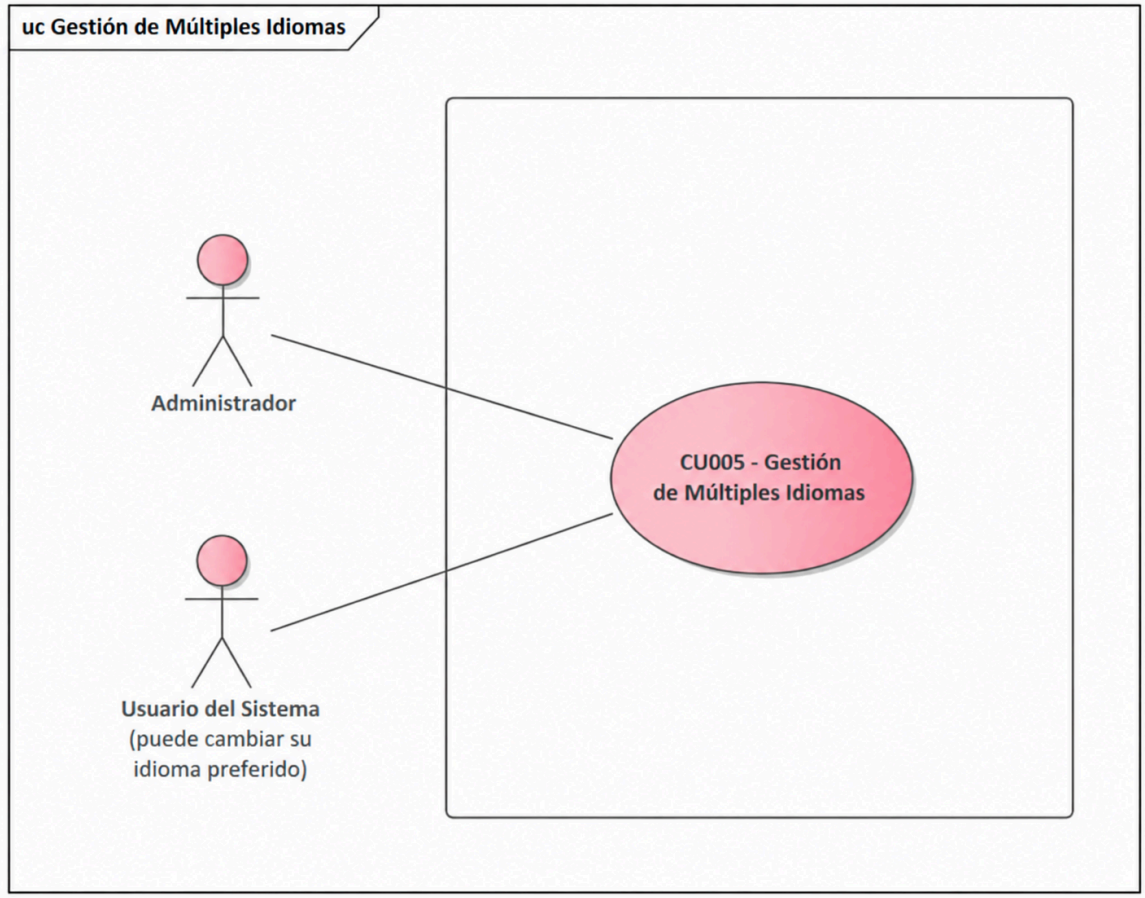
Desde esta pantalla es posible:

- Activar o desactivar idiomas disponibles.

- Modificar traducciones existentes.
- Incorporar nuevos idiomas al sistema.
- Gestionar las traducciones de cada idioma utilizando textos de referencia.

Los cambios realizados se almacenan en la base de datos y quedan disponibles para los usuarios en futuras selecciones de idioma.

Caso de Uso



Especificación de Caso de Uso

ID Y NOMBRE:	CU 005 - Gestión de Múltiples Idiomas

DESCRIPCIÓN: Permite al Administrador gestionar los idiomas disponibles en el sistema (activar, desactivar, crear, renombrar) y editar las traducciones de los textos de la interfaz. También permite a cualquier usuario autenticado cambiar el idioma de su sesión, actualizando automáticamente todos los formularios abiertos de forma instantánea.

ACTOR PRINCIPAL: Administrador

ACTORES SECUNDARIO: Usuario del Sistema (puede cambiar su idioma preferido desde Mi Perfil).

PRECONDICIONES:

- Para gestión de idiomas y traducciones: el usuario debe tener sesión activa con permiso de gestión de usuarios.
- Para cambiar el idioma: cualquier usuario con sesión activa puede hacerlo.
- Debe existir al menos un idioma activo y predeterminado en el sistema.

PUNTOS DE EXTENSIÓN: -

PUNTOS DE INCLUSIÓN: -

ESCENARIO PRINCIPAL:

1. El usuario accede a Administrar → Gestión de Idiomas desde el menú principal.
2. El sistema muestra la lista de idiomas registrados con su estado (activo / inactivo / predeterminado) y la grilla de traducciones del idioma seleccionado.
3. El usuario selecciona un idioma de la lista.
4. El sistema carga los textos del idioma seleccionado en la grilla de traducciones, mostrando la columna de referencia en español para facilitar la tarea.
5. El usuario edita los textos en la columna "Traducción" de la grilla.
6. El usuario presiona "Guardar cambios".
7. El sistema persiste las traducciones editadas. Si el idioma editado es el que está activo

en ese momento, actualiza los textos de todos los formularios abiertos en tiempo real.
FLUJOS ALTERNATIVOS: 3.1 El usuario activa un idioma inactivo: el sistema verifica si tiene traducciones faltantes. Si las tiene, advierte la cantidad y solicita confirmación. Si confirma o no hay faltantes, activa el idioma y lo hace disponible en el selector de idiomas del sistema. 3.2 El usuario desactiva un idioma: el sistema muestra un mensaje de confirmación indicando que los usuarios ya no podrán seleccionarlo. Si confirma, lo desactiva y lo elimina del selector. 3.3 El usuario crea un nuevo idioma: el sistema solicita el código (máx. 5 caracteres, ej: FR) y el nombre (ej: Français). Si el código ya existe, muestra un error. Si no, crea el idioma en estado inactivo para que el administrador cargue sus traducciones antes de activarlo. 4.1 El usuario cambia su idioma preferido desde Mi Perfil o desde el selector del menú: el sistema carga las traducciones del nuevo idioma y notifica a todos los formularios abiertos para que actualicen sus textos de forma inmediata.
POSTCONDICIONES: - Las traducciones editadas quedan guardadas en la base de datos. - Si se cambió el idioma activo, todos los formularios abiertos muestran los textos en el nuevo idioma de forma inmediata. - Si se activó o desactivó un idioma, el selector de idiomas del sistema se actualiza en vivo.

Diagrama de clases

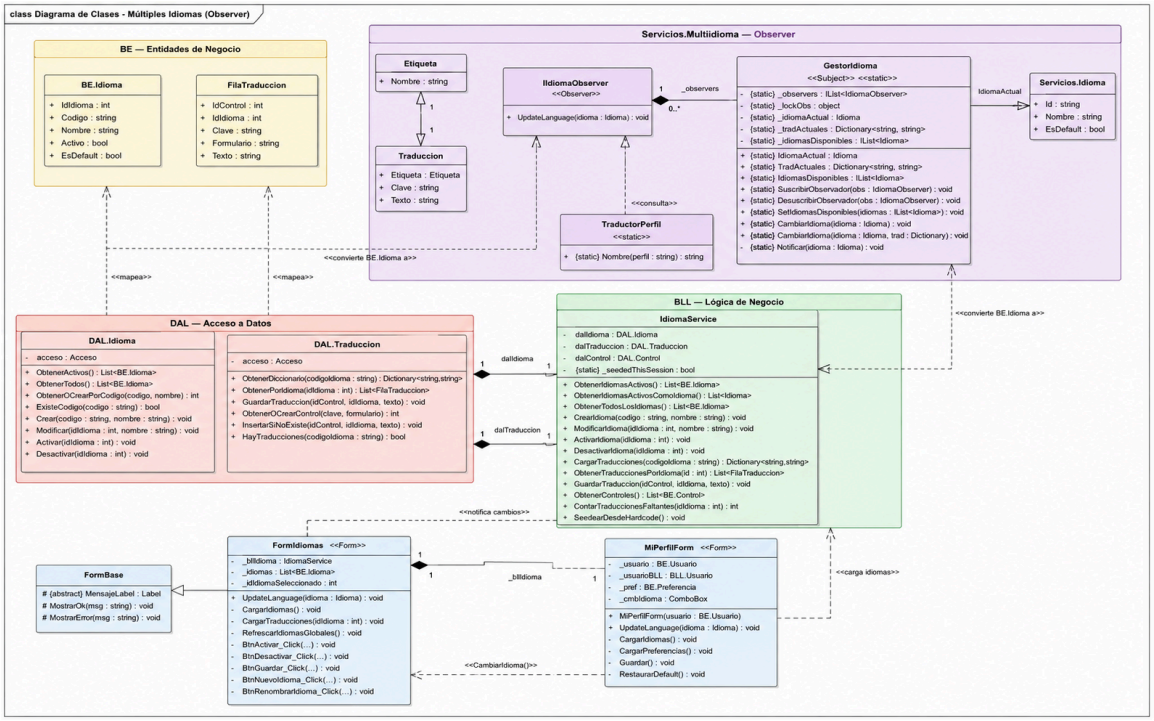
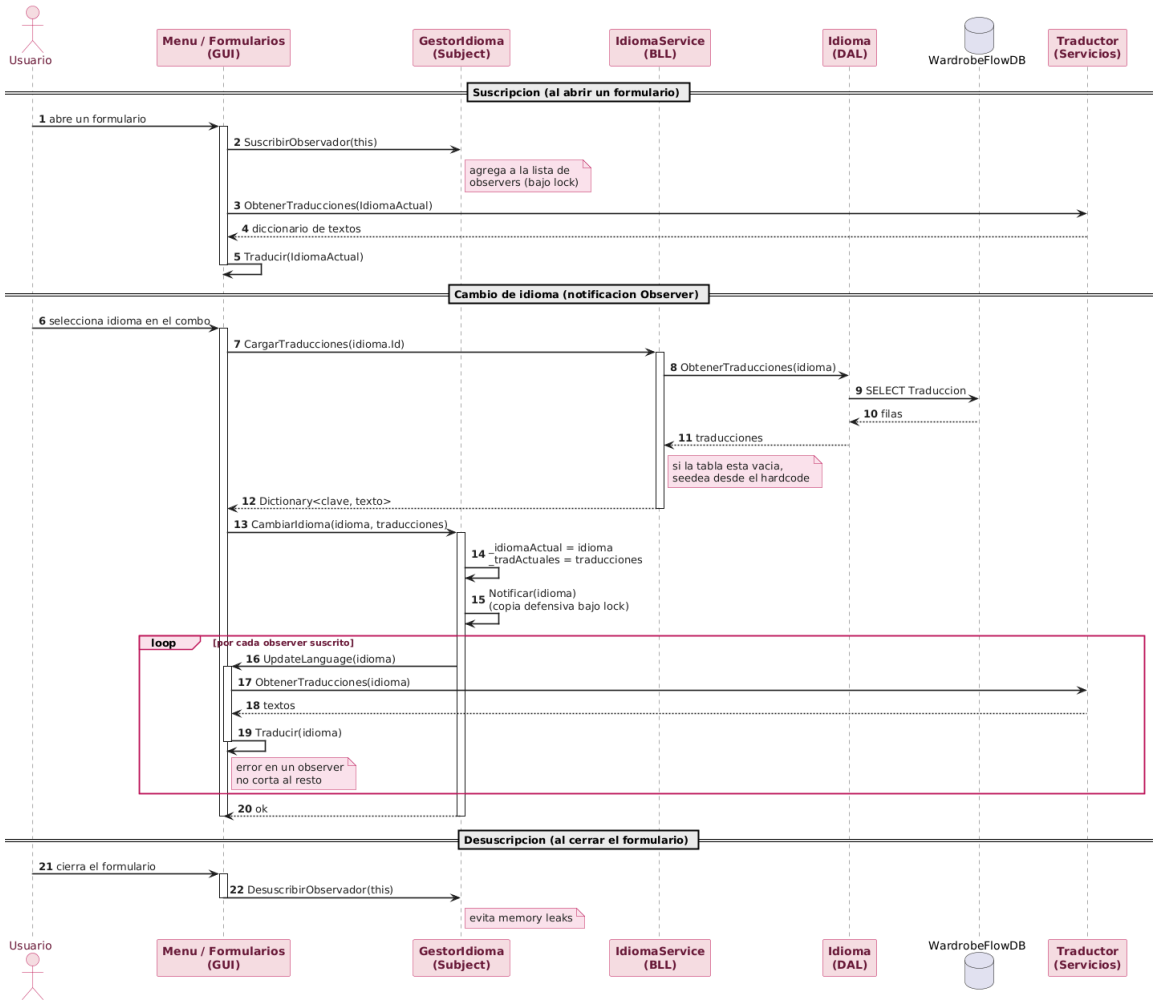
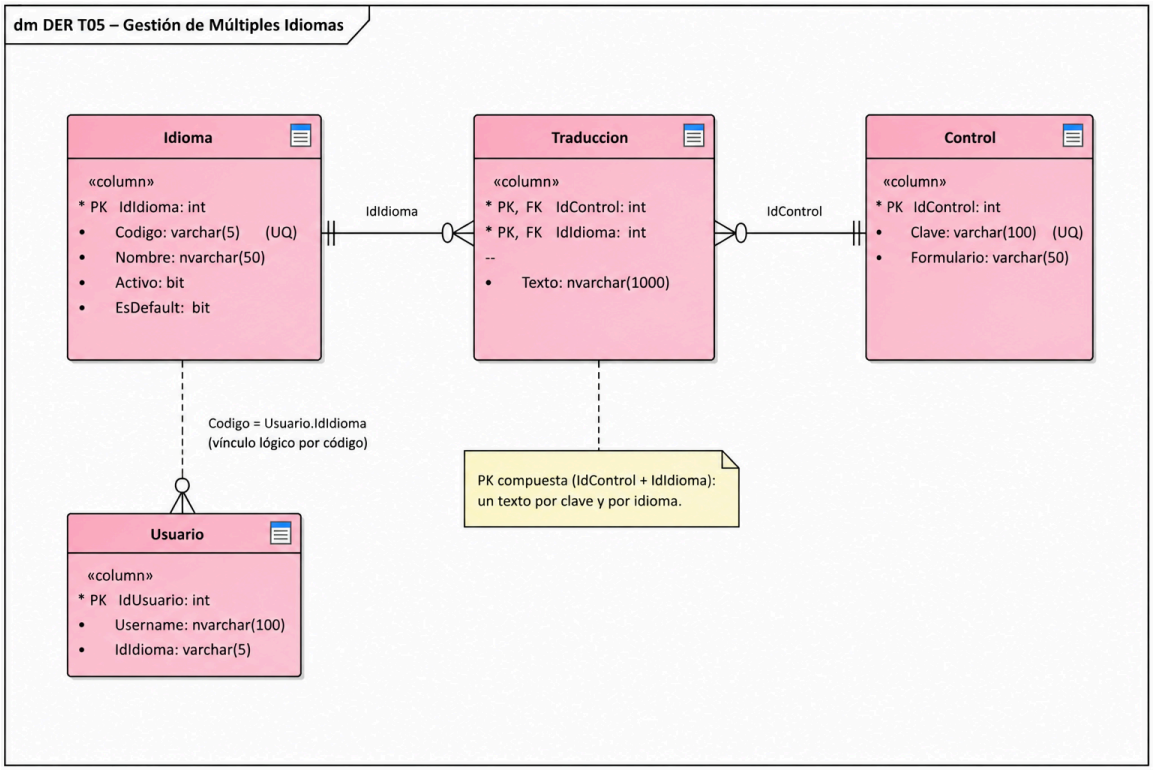


Diagrama de secuencia


Usuario
Menu / Formularios (GUI)
GestorIdioma (Subject)
IdiomaService (BLL)
Idioma (DAL)
WardrobeFlowDB
Traductor (Servicios)
Suscripcion (al abrir un formulario)
1 abre un formulario
2 SuscribirObservador(this)
agrega a la lista de observers (bajo lock)
3 ObtenerTraducciones(IdiomaActual)
4 diccionario de textos
5 Traducir(IdiomaActual)
Cambio de idioma (notificacion Observer)
6 selecciona idioma en el combo
7 CargarTraducciones(idioma.Id)
8 ObtenerTraducciones(idioma)
9 SELECT Traduccion
10 filas
11 traducciones
si la tabla esta vacia, seedea desde el hardcode
12 Dictionary<clave, texto>
13 CambiarIdioma(idioma, traducciones)
14 idiomaActual = idioma
tradActuales = traducciones
15 Notificar(idioma)
(copia defensiva bajo lock)
loop
[por cada observer suscrito]
16 UpdateLanguage(idioma)
17 ObtenerTraducciones(idioma)
18 textos
19 Traducir(idioma)
error en un observer no corta al resto
20 ok
Desuscripcion (al cerrar el formulario)
21 cierra el formulario
22 DesuscribirObservador(this)
evita memory leaks

DER



T06a. Gestión de Bitácora

Objetivo

Registrar de forma cronológica, automática y transparente todas las operaciones relevantes realizadas por los usuarios, permitiendo auditoría completa del sistema.

Descripción detallada de funcionamiento

El sistema cuenta con un registro de auditoría implementado en la clase Servicios.Bitacora, que centraliza tanto la escritura como la consulta de eventos. Cada vez que se ejecuta una acción relevante, el sistema registra automáticamente el usuario que la realizó, la fecha y hora, el módulo involucrado, la actividad ejecutada, su nivel de criticidad y la dirección IP del equipo desde donde se originó.

La clase expone dos métodos de escritura con comportamientos distintos:

- Registrar: requiere sesión activa. Obtiene los datos del usuario desde SessionManager y persiste el evento completo. Si no hay sesión activa, retorna silenciosamente sin lanzar excepción.
- RegistrarSinSesion: no requiere sesión activa. Se usa para eventos previos al login, como intentos fallidos de acceso y bloqueos de cuenta. El idUsuario es opcional (puede ser nulo si se desconoce). Captura cualquier error internamente sin propagarlo.

Para consultas, la GUI accede directamente a Servicios.Bitacora sin pasar por una BLL intermediaria, ya que no aplican reglas de negocio sobre la lectura:

- ObtenerTodos: devuelve todos los registros ordenados por fecha descendente.
- ObtenerUltimosNDias(dias): devuelve los registros de los últimos N días. Normaliza valores menores a 1.
- BuscarPorFiltros(desde, hasta, idUsuario, actividad, criticidad): búsqueda combinada. Cualquier parámetro nulo, vacío o -1 se ignora en el filtro.

La IP local se resuelve en cada registro mediante ObtenerIPLocal(), que obtiene la dirección IPv4 del equipo usando Dns.GetHostEntry.

Descripción de métodos

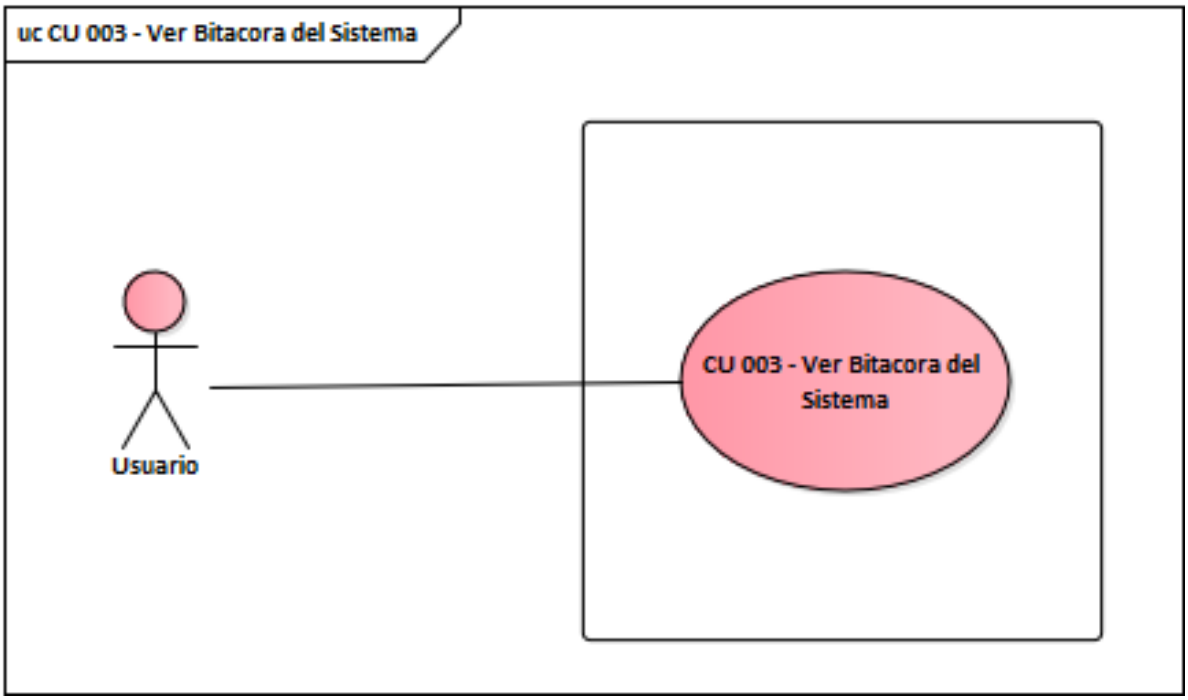
Método	Descripción	Acceso
Registrar(modulo, actividad, criticidad)	Registra un evento con sesión activa. Captura usuario e IP automáticamente.	Administrador
RegistrarSinSesion (modulo, actividad, criticidad, idUsuario?, detalle?)	Registra un evento sin sesión activa. Usado para intentos fallidos y bloqueos.	Administrador
ObtenerTodos()	Devuelve todos los registros de la bitácora ordenados por fecha descendente.	Administrador
ObtenerUltimosNDias(dias)	Devuelve los registros de los últimos N días.	Administrador

BuscarPorFiltros(desde, hasta, idUsuario, actividad, criticidad)	Búsqueda combinada con filtros opcionales.	Administrador
ObtenerIPLocal()	Resuelve la IPv4 local del equipo para incluirla en cada registro.	Administrador

Niveles de criticidad (BE.Criticidad)

Valor	Nombre	Uso
0	None	Login / Logout exitoso
1	Baja	Lecturas y navegación
2	Media	Altas y modificaciones
3	Alta	Eliminaciones y operaciones privilegiadas
4	IntentosLogin	Intentos fallidos de acceso
5	RecuperacionClave	Reset de contraseña
6	BloqueosCuenta	Bloqueo automático de cuenta

Caso de Uso



Especificación de Caso de Uso

ID Y NOMBRE:	CU 006 - Control de Cambios de Usuarios
DESCRIPCIÓN: Permite al Administrador consultar el historial de versiones de un usuario y restaurar su estado a una versión anterior, revirtiendo contraseña y estado de bloqueo. Antes de cada operación destructiva (reset de clave, desbloqueo, archivado) el sistema guarda automáticamente una copia del estado actual del usuario, garantizando que cualquier cambio pueda deshacerse.	
ACTOR PRINCIPAL: Administrador	
ACTORES SECUNDARIO: -	

PRECONDICIONES:

- El usuario debe tener una sesión activa con permiso de gestión de usuarios.
- Debe existir al menos un registro de historial en el sistema.

PUNTOS DE EXTENSIÓN: -

PUNTOS DE INCLUSIÓN: -

ESCENARIO PRINCIPAL:

1. El usuario accede a Administrar → Historial de Usuarios desde el menú principal.
2. El sistema muestra la lista de usuarios disponibles.
3. El usuario selecciona un usuario y presiona "Cargar".
4. El sistema muestra la grilla con el historial de versiones del usuario seleccionado: fecha, quién realizó el cambio, descripción del cambio y estado (Activo / Bloqueado).
5. El usuario selecciona la versión a la que desea restaurar el estado del usuario.
6. El usuario presiona "Restaurar Versión Seleccionada".
7. El sistema muestra un diálogo de confirmación con los datos del estado a restaurar: usuario afectado, fecha del snapshot y descripción.
8. El usuario confirma la restauración.
9. El sistema guarda el estado actual del usuario como nueva versión (copia de seguridad previa).
10. El sistema aplica el estado del snapshot seleccionado sobre el usuario: restaura contraseña, estado de bloqueo e intentos fallidos.
11. El sistema registra la operación en la bitácora con criticidad alta y recarga la grilla mostrando la nueva versión generada.

FLUJOS ALTERNATIVOS:

4.1 El usuario seleccionado no tiene historial registrado: el sistema muestra la grilla vacía y el botón "Restaurar" permanece deshabilitado.

8.1 El usuario no confirma la restauración: el sistema cancela la operación y el estado del usuario no cambia.

9.1 El sistema no puede guardar la copia de seguridad previa por un error en la base de datos: la operación se cancela completamente y el estado del usuario no se modifica. El sistema muestra un mensaje de error indicando que no fue posible proceder de forma segura.

POSTCONDICIONES:

- El usuario afectado tiene su contraseña, estado de bloqueo e intentos fallidos restaurados al estado del snapshot seleccionado.
- El historial del usuario registra tanto la copia de seguridad previa como la operación de restauración.
- La operación queda registrada en la bitácora del sistema.

Diagrama de Clases

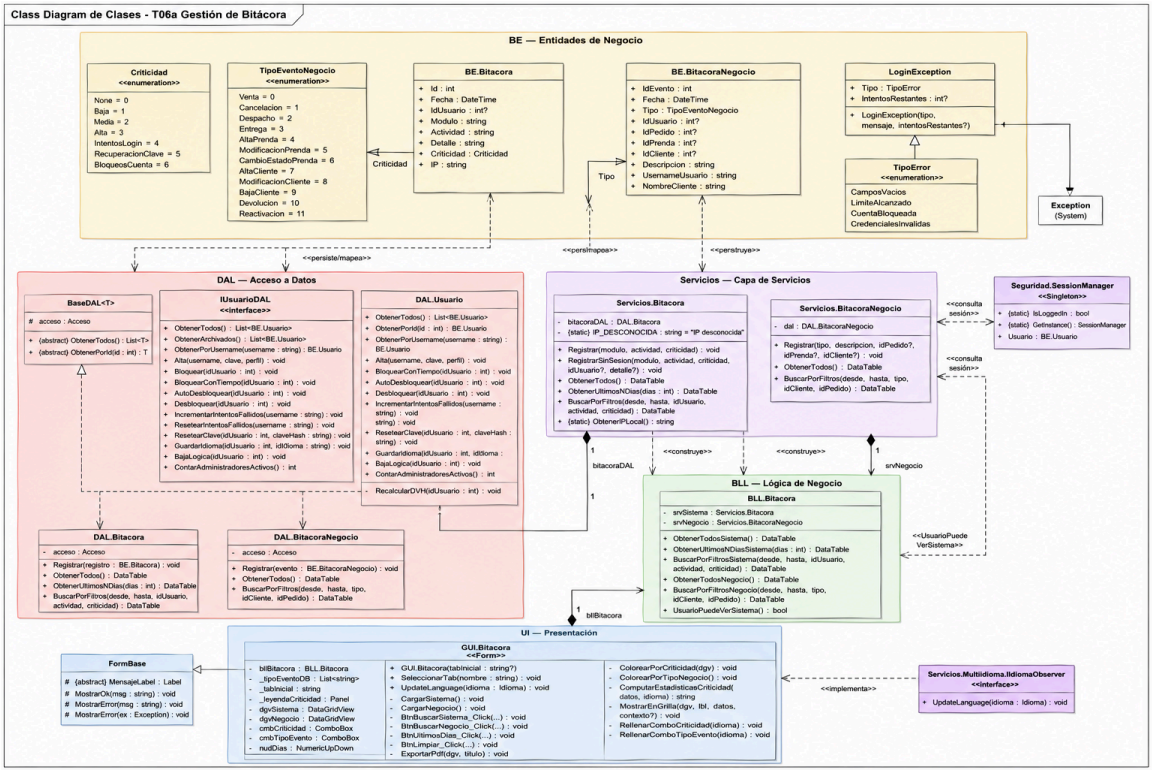
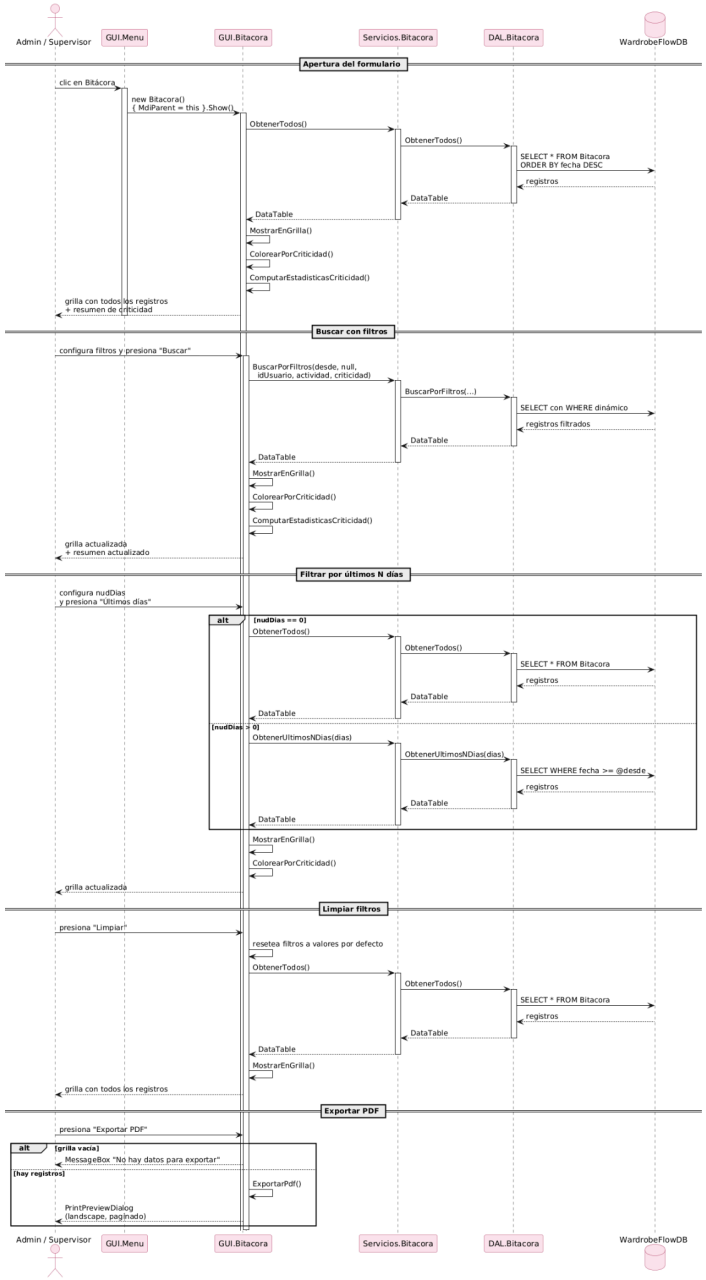
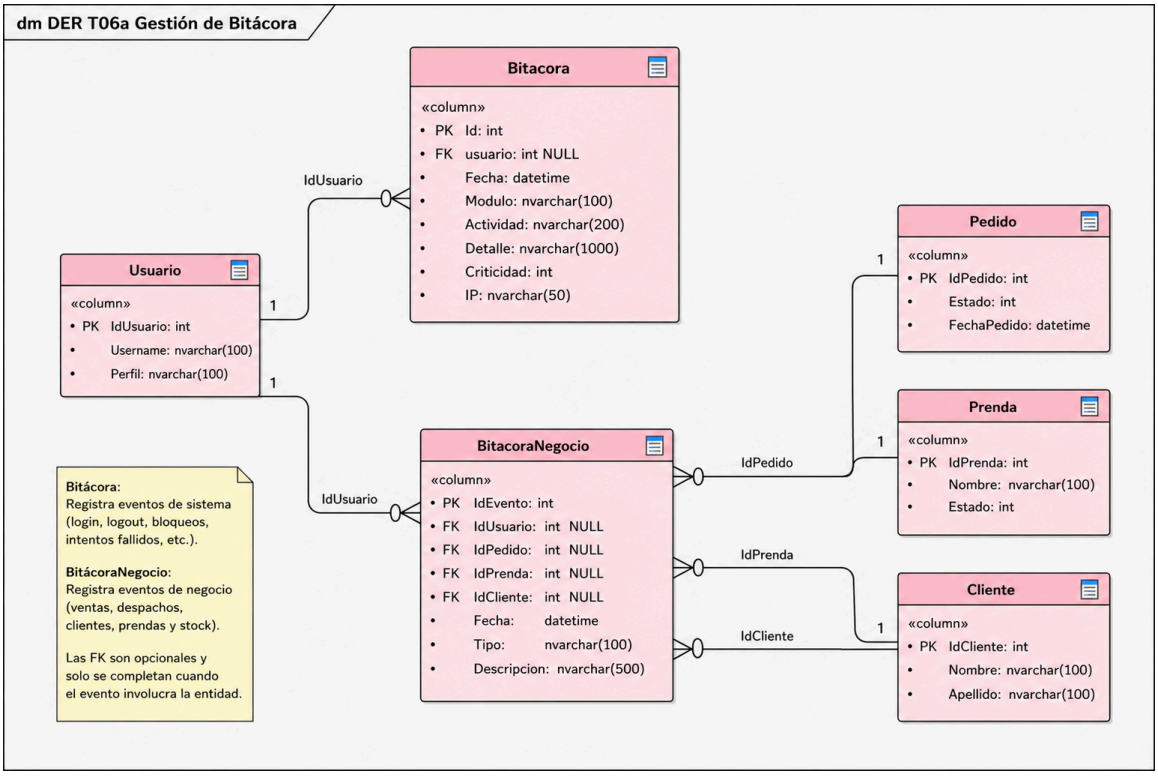

Class Diagram de Clases - T06a Gestión de Bitácora
BE — Entidades de Negocio
Criticidad
<<enumeration>>
None = 0
Baja = 1
Media = 2
Alta = 3
IntentosLogin = 4
RecuperacionClave = 5
BloqueosCuenta = 6
TipoEventoNegocio
<<enumeration>>
Venta = 0
Cancelacion = 1
Despacho = 2
Entrega = 3
AltaPrenda = 4
ModificacionPrenda = 5
CambioEstadoPrenda = 6
AltaCliente = 7
ModificacionCliente = 8
BajaCliente = 9
Devolucion = 10
Reactivacion = 11
BE.Bitacora
+ Id : int
+ Fecha : DateTime
+ IdUsuario : int?
+ Modulo : string
+ Actividad : string
+ Detalle : string
+ Criticidad : Criticidad
+ IP : string
Criticidad
Tipo
BE.BitacoraNegocio
+ IdEvento : int
+ Fecha : DateTime
+ Tipo : TipoEventoNegocio
+ IdUsuario : int?
+ IdPedido : int?
+ IdPrenda : int?
+ IdCliente : int?
+ Descripcion : string
+ UsernameUsuario : string
+ NombreCliente : string
LoginException
+ Tipo : TipoError
+ IntentosRestantes : int?
+ LoginException(tipo, mensaje, intentosRestantes?)
TipoError
<<enumeration>>
CamposVacios
LimiteAlcanzado
CuentaBloqueada
CredencialesInvalidas
Exception (System)
<<persiste/mapea>>
<<persiste/mapea>>
<<persiste/mapea>>
DAL — Acceso a Datos
BaseDAL<T>
acceso : Acceso
+ {abstract} ObtenerTodos() : List<T>
+ {abstract} ObtenerPorId(id : int) : T
IUsuarioDAL
<<interface>>
+ ObtenerTodos() : List<BE.Usuario>
+ ObtenerArchivados() : List<BE.Usuario>
+ ObtenerPorUsername(username : string) : BE.Usuario
+ Alta(username, clave, perfil) : void
+ Bloquear(idUsuario : int) : void
+ BloquearConTiempo(idUsuario : int) : void
+ AutoDesbloquear(idUsuario : int) : void
+ Desbloquear(idUsuario : int) : void
+ IncrementarIntentosFallidos(username : string) : void
+ ResetearIntentosFallidos(username : string) : void
+ ResetearClave(idUsuario : int, claveHash : string) : void
+ GuardarIdioma(idUsuario : int, idIdioma : string) : void
+ BajaLogica(idUsuario : int) : void
+ ContarAdministradoresActivos() : int
DAL.Usuario
+ ObtenerTodos() : List<BE.Usuario>
+ ObtenerPorId(id : int) : BE.Usuario
+ ObtenerPorUsername(username : string) : BE.Usuario
+ Alta(username, clave, perfil) : void
+ BloquearConTiempo(idUsuario : int) : void
+ AutoDesbloquear(idUsuario : int) : void
+ Desbloquear(idUsuario : int) : void
+ IncrementarIntentosFallidos(username : string) : void
+ ResetearClave(idUsuario : int, claveHash : string) : void
+ GuardarIdioma(idUsuario : int, idIdioma : string) : void
+ BajaLogica(idUsuario : int) : void
+ ContarAdministradoresActivos() : int
- RecalcularDVH(idUsuario : int) : void
DAL.Bitacora
- acceso : Acceso
+ Registrar(registro : BE.Bitacora) : void
+ ObtenerTodos() : DataTable
+ ObtenerUltimosNDias(dias : int) : DataTable
+ BuscarPorFiltros(desde, hasta, idUsuario, actividad, criticidad) : DataTable
DAL.BitacoraNegocio
- acceso : Acceso
+ Registrar(evento : BE.BitacoraNegocio) : void
+ ObtenerTodos() : DataTable
+ BuscarPorFiltros(desde, hasta, tipo, idCliente, idPedido) : DataTable
Servicios — Capa de Servicios
Servicios.Bitacora
- bitacoraDAL : DAL.Bitacora
- {static} IP_DESCONOCIDA : string = "IP desconocida"
+ Registrar(modulo, actividad, criticidad) : void
+ RegistrarSinSesion(modulo, actividad, criticidad, idUsuario?, detalle?) : void
+ ObtenerTodos() : DataTable
+ ObtenerUltimosNDias(dias : int) : DataTable
+ BuscarPorFiltros(desde, hasta, idUsuario, actividad, criticidad) : DataTable
+ {static} ObtenerIPLocal() : string
Servicios.BitacoraNegocio
- dal : DAL.BitacoraNegocio
+ Registrar(tipo, descripcion, idPedido?, idPrenda?, idCliente?) : void
+ ObtenerTodos() : DataTable
+ BuscarPorFiltros(desde, hasta, tipo, idCliente, idPedido) : DataTable
<<consulta sesión>>
<<consulta sesión>>
Seguridad.SessionManager
<<Singleton>>
+ {static} IsLoggedIn : bool
+ {static} GetInstance() : SessionManager
+ Usuario : BE.Usuario
1
bitacoraDAL
<<construye>>
<<construye>>
1
srvNegocio
BLL — Lógica de Negocio
BLL.Bitacora
- srvSistema : Servicios.Bitacora
- srvNegocio : Servicios.BitacoraNegocio
+ ObtenerTodosSistema() : DataTable
+ ObtenerUltimosNDiasSistema(dias : int) : DataTable
+ BuscarPorFiltrosSistema(desde, hasta, idUsuario, actividad, criticidad) : DataTable
+ ObtenerTodosNegocio() : DataTable
+ BuscarPorFiltrosNegocio(desde, hasta, tipo, idCliente, idPedido) : DataTable
+ UsuarioPuedeVerSistema() : bool
<<UsuarioPuede VerSistema>>
1
blBitacora
UI — Presentación
FormBase
{abstract} MensajeLabel : Label
MostrarOk(msg : string) : void
MostrarError(msg : string) : void
MostrarError(ex : Exception) : void
GUI.Bitacora
<<Form>>
- blBitacora : BLL.Bitacora
- _tipoEventoDB : List<string>
- _tabInicial : string
- _leyendaCriticidad : Panel
- dgvSistema : DataGridView
- dgvNegocio : DataGridView
- cmbCriticidad : ComboBox
- cmbTipoEvento : ComboBox
- nudDias : NumericUpDown
+ GUI.Bitacora(tabInicial : string?)
+ SeleccionarTab(nombre : string) : void
+ UpdateLanguage(idioma : Idioma) : void
- CargarSistema() : void
- CargarNegocio() : void
- BtnBuscarSistema_Click(...) : void
- BtnBuscarNegocio_Click(...) : void
- BtnUltimosDias_Click(...) : void
- BtnLimpiar_Click(...) : void
- ExportarPdf(dgv, titulo) : void
- ColorearPorCriticidad(dgv) : void
- ColorearPorTipoNegocio() : void
- ComputarEstadisticasCriticidad(datos, idioma) : string
- MostrarEnGrilla(dgv, lbl, datos, contexto?) : void
- RellenarComboCriticidad(idioma) : void
- RellenarComboTipoEvento(idioma) : void
<<implementa>>
Servicios.MultiIdioma.IIdiomaObserver
<<interface>>
+ UpdateLanguage(idioma : Idioma) : void

Diagrama de Secuencia



DER



T06b. Control de Cambios

Objetivo

Garantizar la trazabilidad y la posibilidad de revertir cambios realizados sobre la entidad más sensible del sistema: el Usuario. El sistema debe permitir conocer quién realizó una modificación, cuándo se efectuó y cuál era el estado previo del usuario, de manera que operaciones como el reseteo de contraseña, el desbloqueo de cuenta o una restauración puedan deshacerse recuperando una versión anterior.

Descripción funcional

El control de cambios se implementa utilizando el patrón Memento (GoF), que permite guardar el estado de un objeto sin exponer sus detalles internos y restaurarlo posteriormente cuando sea necesario.

Estructura de componentes

La solución se compone de los siguientes elementos:

- BE.Usuario: representa al usuario y es responsable de generar y restaurar su propio estado.
- BE.VersionUsuario: almacena una versión histórica del usuario, incluyendo información relevante como nombre de usuario, contraseña cifrada, estado de la cuenta, intentos de acceso y metadatos de auditoría.
- BLL.CuidadorHistorial: administra el almacenamiento y recuperación de las versiones históricas.
- BLL.VersionUsuario: coordina el proceso de captura y restauración de versiones, actuando como punto de acceso para la lógica de negocio y la interfaz de usuario.

Funcionamiento del control de cambios

Captura de versiones

Antes de realizar una operación que modifique información sensible del usuario, el sistema guarda automáticamente una versión del estado actual. De esta forma, siempre existe un punto de recuperación previo al cambio realizado.

Por ejemplo, antes de resetear una contraseña o desbloquear una cuenta, se registra una nueva versión histórica y luego se ejecuta la operación correspondiente.

Restauración de versiones

El sistema permite seleccionar una versión histórica y restaurar el usuario a ese estado anterior. Antes de efectuar la restauración, también se guarda el estado actual, lo que posibilita revertir posteriormente la propia restauración si fuera necesario.

Durante este proceso se validan las condiciones de seguridad correspondientes y se registra la operación en la bitácora del sistema con nivel de criticidad alto.

Interfaz de usuario

La funcionalidad se encuentra disponible en VersionHistorialForm, donde el administrador puede:

- Seleccionar un usuario.
- Visualizar el historial de versiones registradas.
- Consultar fecha, actor y detalle de cada cambio.
- Restaurar una versión previa mediante una acción confirmada.

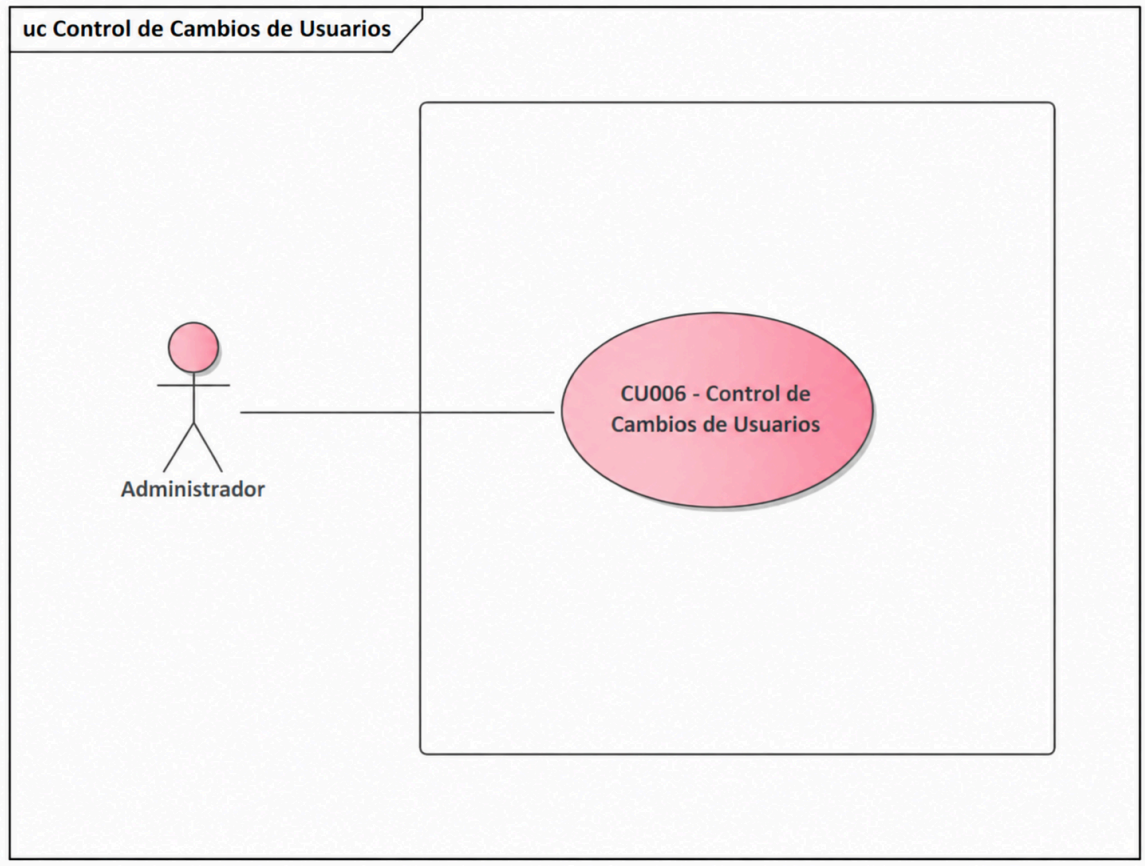
Además, el formulario implementa el sistema de multiidioma de la aplicación, permitiendo que toda la interfaz y los mensajes se adapten dinámicamente al idioma seleccionado.

Beneficios de la solución

Esta implementación aporta:

- Trazabilidad completa de las modificaciones realizadas sobre usuarios.
- Posibilidad de auditoría y recuperación ante errores o cambios no deseados.
- Mayor seguridad y control sobre operaciones sensibles del sistema.
- Separación clara entre la captura del estado y su almacenamiento, gracias al uso del patrón Memento.

Caso de Uso



Especificación de Caso de Uso

ID Y NOMBRE:	CU 006 - Control de Cambios de Usuarios
DESCRIPCIÓN: Permite al Administrador consultar el historial de versiones de un usuario y restaurar su estado a una versión anterior, revirtiendo contraseña y estado de bloqueo. Antes de cada operación destructiva (reset de clave, desbloqueo, archivado) el sistema guarda automáticamente una copia del estado actual del usuario, garantizando que cualquier cambio pueda deshacerse.	
ACTOR PRINCIPAL: Administrador	

ACTORES SECUNDARIO: -

PRECONDICIONES:

- El usuario debe tener una sesión activa con permiso de gestión de usuarios.
- Debe existir al menos un registro de historial en el sistema.

PUNTOS DE EXTENSIÓN: -

PUNTOS DE INCLUSIÓN: -

ESCENARIO PRINCIPAL:

1. El usuario accede a Administrar → Historial de Usuarios desde el menú principal.
2. El sistema muestra la lista de usuarios disponibles.
3. El usuario selecciona un usuario y presiona "Cargar".
4. El sistema muestra la grilla con el historial de versiones del usuario seleccionado: fecha, quién realizó el cambio, descripción del cambio y estado (Activo / Bloqueado).
5. El usuario selecciona la versión a la que desea restaurar el estado del usuario.
6. El usuario presiona "Restaurar Versión Seleccionada".
7. El sistema muestra un diálogo de confirmación con los datos del estado a restaurar: usuario afectado, fecha del snapshot y descripción.
8. El usuario confirma la restauración.
9. El sistema guarda el estado actual del usuario como nueva versión (copia de seguridad previa).
10. El sistema aplica el estado del snapshot seleccionado sobre el usuario: restaura contraseña, estado de bloqueo e intentos fallidos.
11. El sistema registra la operación en la bitácora con criticidad alta y recarga la grilla mostrando la nueva versión generada.

FLUJOS ALTERNATIVOS:

4.1 El usuario seleccionado no tiene historial registrado: el sistema muestra la grilla vacía y el botón "Restaurar" permanece deshabilitado.

8.1 El usuario no confirma la restauración: el sistema cancela la operación y el estado del usuario no cambia.

9.1 El sistema no puede guardar la copia de seguridad previa por un error en la base de datos: la operación se cancela completamente y el estado del usuario no se modifica. El sistema muestra un mensaje de error indicando que no fue posible proceder de forma segura.

POSTCONDICIONES:

- El usuario afectado tiene su contraseña, estado de bloqueo e intentos fallidos restaurados al estado del snapshot seleccionado.
- El historial del usuario registra tanto la copia de seguridad previa como la operación de restauración.
- La operación queda registrada en la bitácora del sistema.

Diagrama de clases

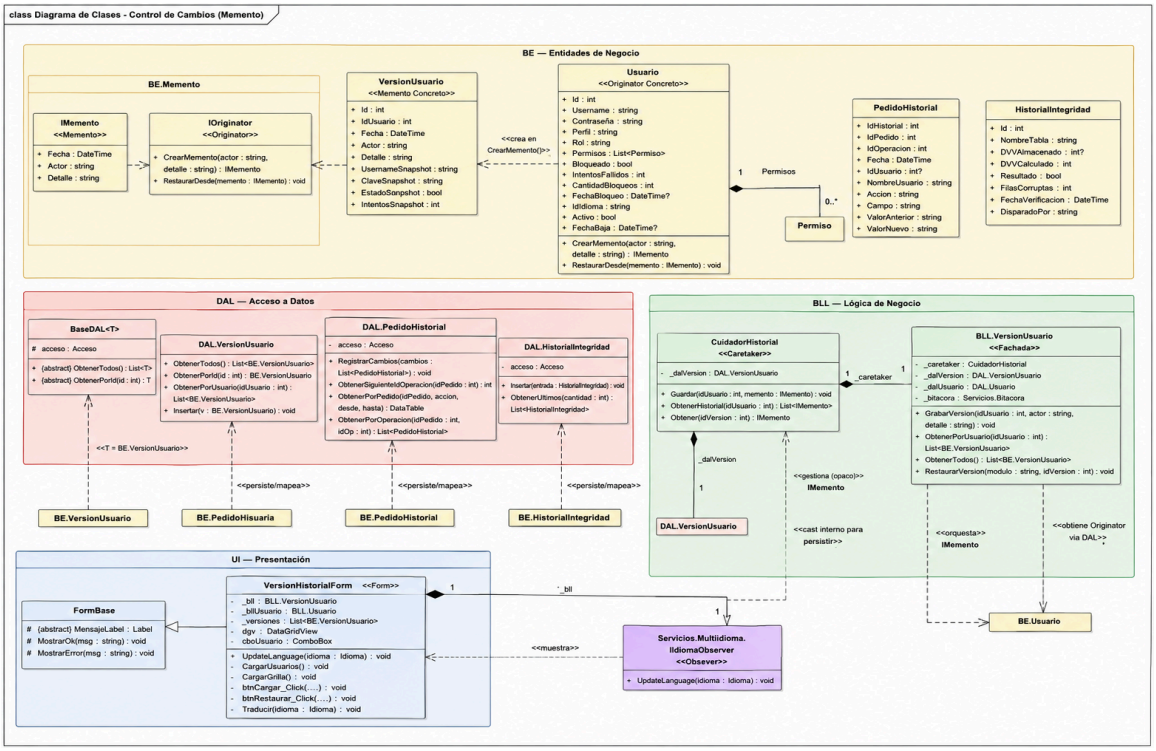
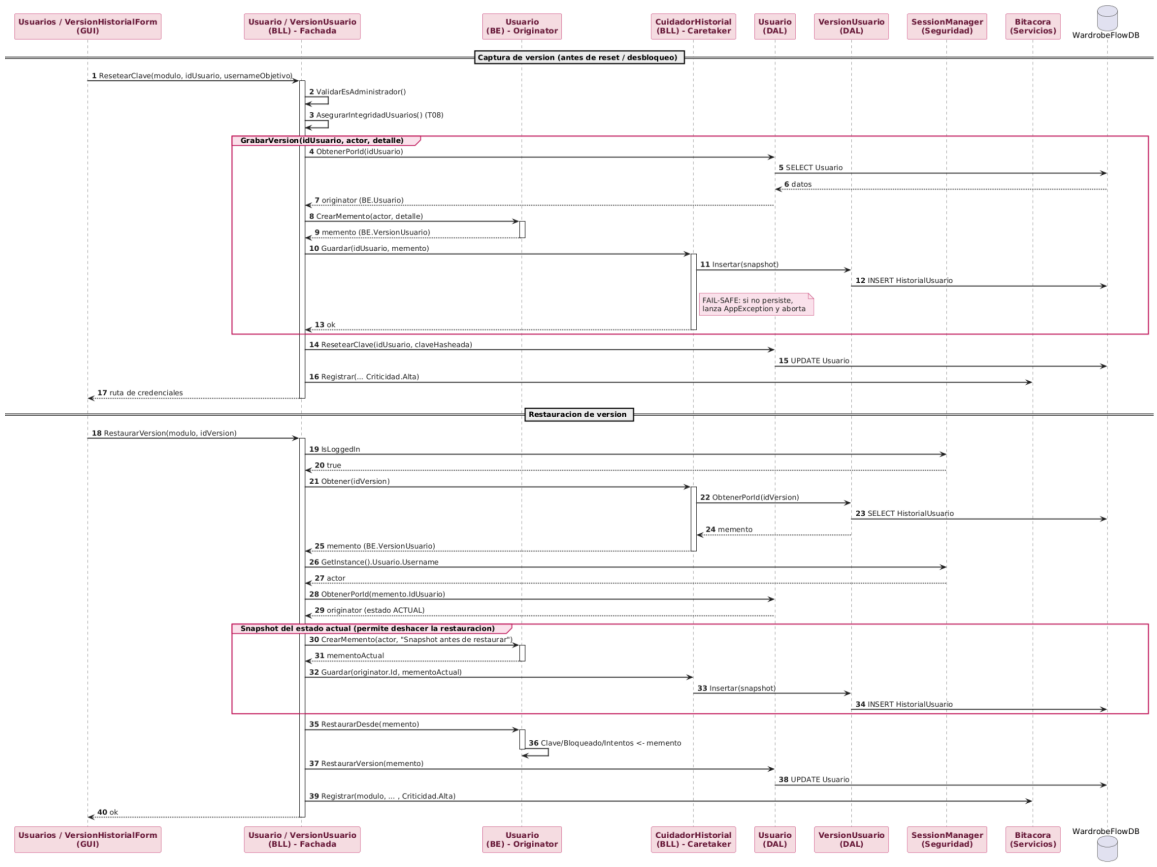
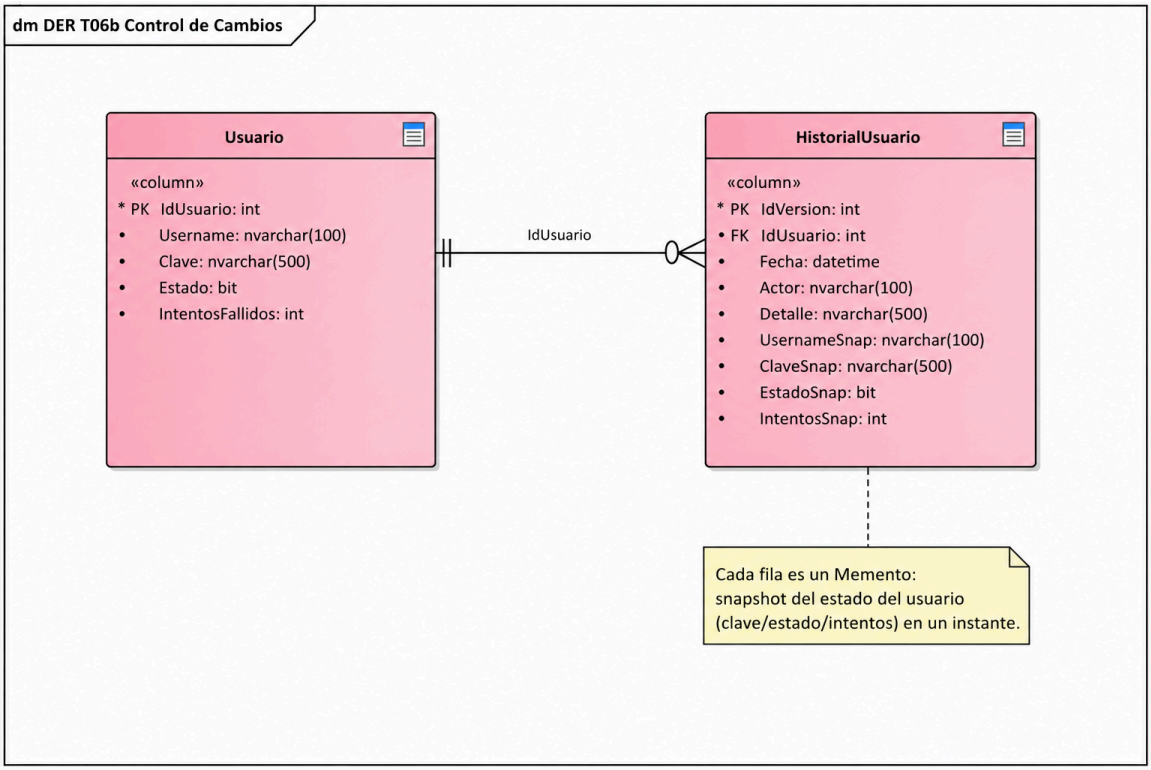


Diagrama de secuencia



DER



T07. Gestión de Backup y Restauración

Objetivo

Permitir generar, administrar y restaurar copias de seguridad de la base de datos, garantizando la recuperabilidad de la información ante fallas o situaciones excepcionales. Cada backup se cifra con AES-128 y contraseña personal, se verifica la integridad de la BD antes de respaldar, y toda operación queda registrada en la bitácora del sistema.

Descripción funcional

El módulo de Gestión de Backups tiene como objetivo garantizar la disponibilidad y recuperación de la información almacenada por el sistema ante eventos de pérdida,

corrupción o necesidad de restauración de datos históricos. Su acceso se encuentra restringido exclusivamente a usuarios con perfil Administrador y puede ser invocado desde la opción Administrar → Backup.

La interfaz presenta un catálogo de los archivos de respaldo almacenados en la carpeta de backups de la aplicación, mostrando para cada uno su nombre, fecha de generación, usuario responsable de la operación y tamaño del archivo. Los registros se visualizan ordenados cronológicamente desde el respaldo más reciente hacia el más antiguo.

El módulo permite realizar las siguientes operaciones:

Generación de Backup

La generación de un respaldo requiere, como condición previa, la verificación satisfactoria de los mecanismos de integridad implementados mediante Dígito Verificador Horizontal (DVH) y Dígito Verificador Vertical (DVV). En caso de detectarse inconsistencias, el proceso es cancelado para evitar la creación de copias de seguridad sobre una base de datos comprometida.

Una vez validada la integridad, el sistema solicita al usuario una contraseña de cifrado con doble confirmación. El respaldo generado es protegido mediante un mecanismo de cifrado basado en PBKDF2 y almacenado con extensión *.wfbak*. El nombre del archivo sigue la convención: **WardrobeFlow_Backup_yyyyMMddHHmmss_USUARIO.wfbak** donde se incluye la fecha, hora y usuario responsable de la generación del respaldo.

Backup de Instalación Limpia

Además de los respaldos operativos habituales, el sistema permite generar un backup de instalación limpia. Este respaldo constituye un punto de restauración inicial de referencia y puede ser utilizado para retornar el sistema a un estado base previamente validado.

El procedimiento de generación es idéntico al utilizado para los backups convencionales, incluyendo la validación de integridad y el cifrado del archivo. La diferencia radica en que el nombre del respaldo incorpora un identificador especial que permite distinguirlo como respaldo inicial.

Restauración de Backup

La funcionalidad de restauración permite recuperar el estado de la base de datos a partir de un archivo de respaldo previamente generado.

Antes de ejecutar la restauración, el sistema analiza los metadatos del archivo seleccionado para informar al usuario la fecha de creación del respaldo y estimar el período de información que podría perderse como consecuencia de la operación. Si el archivo se encuentra protegido mediante cifrado, se solicita la contraseña correspondiente para proceder a su descifrado.

Como medida de seguridad adicional, el sistema ejecuta una validación de consistencia utilizando el comando RESTORE VERIFYONLY, garantizando que el archivo de respaldo sea válido y restaurable. Posteriormente, todas las conexiones activas son finalizadas mediante la configuración temporal de la base de datos en modo SINGLE_USER, tras lo cual se ejecuta la restauración utilizando la opción WITH REPLACE. Finalizado el proceso, la aplicación reinicia su ejecución para asegurar la correcta carga de la información restaurada.

Importación de Backups Externos

El módulo permite importar archivos de respaldo provenientes de ubicaciones externas al repositorio de backups de la aplicación. Se admiten archivos con extensión *.wfbak* y *.bak*. Una vez seleccionado el archivo, el sistema ejecuta el mismo flujo de validación, descifrado (cuando corresponda), verificación de integridad y restauración utilizado para los respaldos internos.

Eliminación de Backups

La funcionalidad de eliminación permite remover respaldos almacenados en el catálogo del sistema. Antes de efectuar la operación, se solicita una confirmación explícita al usuario administrador. Confirmada la acción, el archivo es eliminado físicamente del sistema de archivos.

Auditoría de Operaciones

Con el propósito de garantizar la trazabilidad y el cumplimiento de los requisitos de seguridad, todas las operaciones relacionadas con la gestión de respaldos son registradas en la Bitácora del sistema. Esto incluye la generación de backups, restauraciones, importaciones y eliminaciones. Cada evento es almacenado con nivel de criticidad Alta, registrando la fecha y hora de ejecución, el usuario responsable y el detalle de la acción realizada.

Situación	Comportamiento
BD con DVN/DVV corrupto	Se aborta el backup con mensaje explicativo.
Contraseñas no coinciden al crear	Se pide reingreso; no se genera el archivo.
Contraseñas no coinciden al restaurar	AppException: "La contraseña del backup es incorrecta o el archivo está dañado"
Contraseña incorrecta al restaurar (VERIFYONLY falla)	AppException: "El archivo es inválido o está corrupto y no puede restaurarse"
Usuario sin perfil Administrador	AppException: "Solo un Administrador puede realizar operaciones de backup"

Diagrama de Clases

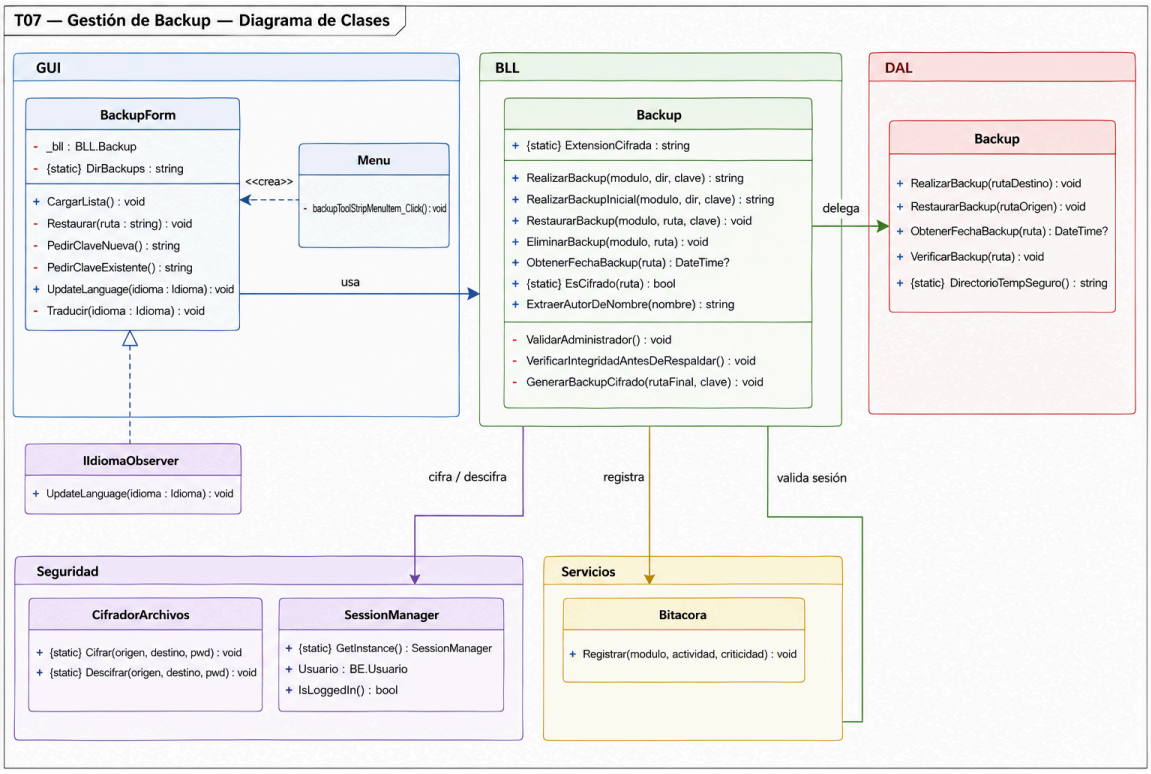
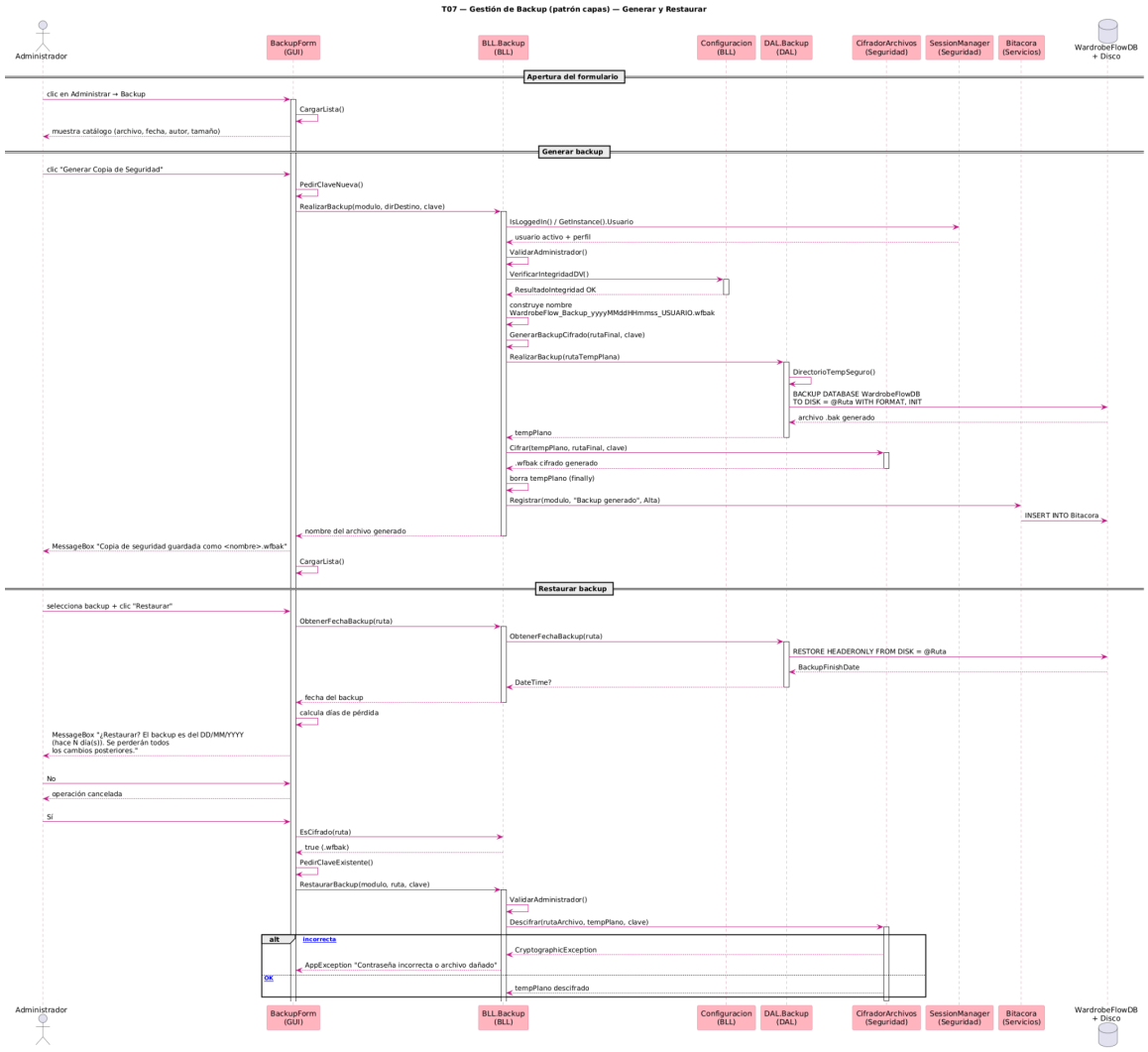


Diagrama de secuencia



T08. Dígitos Verificadores

Objetivo

Garantizar la integridad de la información almacenada en la base de datos mediante la detección de modificaciones realizadas por fuera de la aplicación, como alteraciones

directas en SQL Server, inserción o eliminación de registros y cambios no autorizados sobre los datos.

La solución se aplica principalmente sobre la entidad Usuario y se extiende a otras entidades críticas del sistema, como Cliente, Empleado y Pedido.

Descripción funcional

El sistema implementa un mecanismo de control de integridad basado en dígitos verificadores. Su finalidad es detectar inconsistencias o manipulaciones externas que no hayan sido realizadas a través de las funcionalidades previstas por la aplicación. Para ello se utilizan dos niveles de verificación:

DVH (Dígito Verificador Horizontal)

El DVH se calcula para cada registro individual y se almacena junto a los datos correspondientes. Su función es verificar la integridad de la información contenida en una fila, permitiendo detectar modificaciones realizadas sobre cualquiera de sus campos.

DVV (Dígito Verificador Vertical)

El DVV se calcula considerando el conjunto de registros de una tabla y se almacena de forma independiente. Este mecanismo permite detectar situaciones como inserciones, eliminaciones o alteraciones de registros que puedan comprometer la consistencia general de la información.

Proceso de verificación

Antes de permitir el acceso al sistema, se ejecuta una validación de integridad sobre las entidades protegidas. Durante este proceso, el sistema recalcula los dígitos verificadores utilizando los datos almacenados en la base de datos y los compara con los valores previamente registrados. Si se detecta alguna discrepancia, se considera que la integridad de la información ha sido comprometida y el acceso al sistema es bloqueado hasta que un administrador intervenga. Dependiendo del escenario, el administrador puede recalcular los dígitos verificadores o restaurar la información desde una copia de seguridad válida.

Consideraciones de implementación

El algoritmo de cálculo fue diseñado para minimizar la posibilidad de colisiones y aumentar la capacidad de detección de modificaciones no autorizadas.

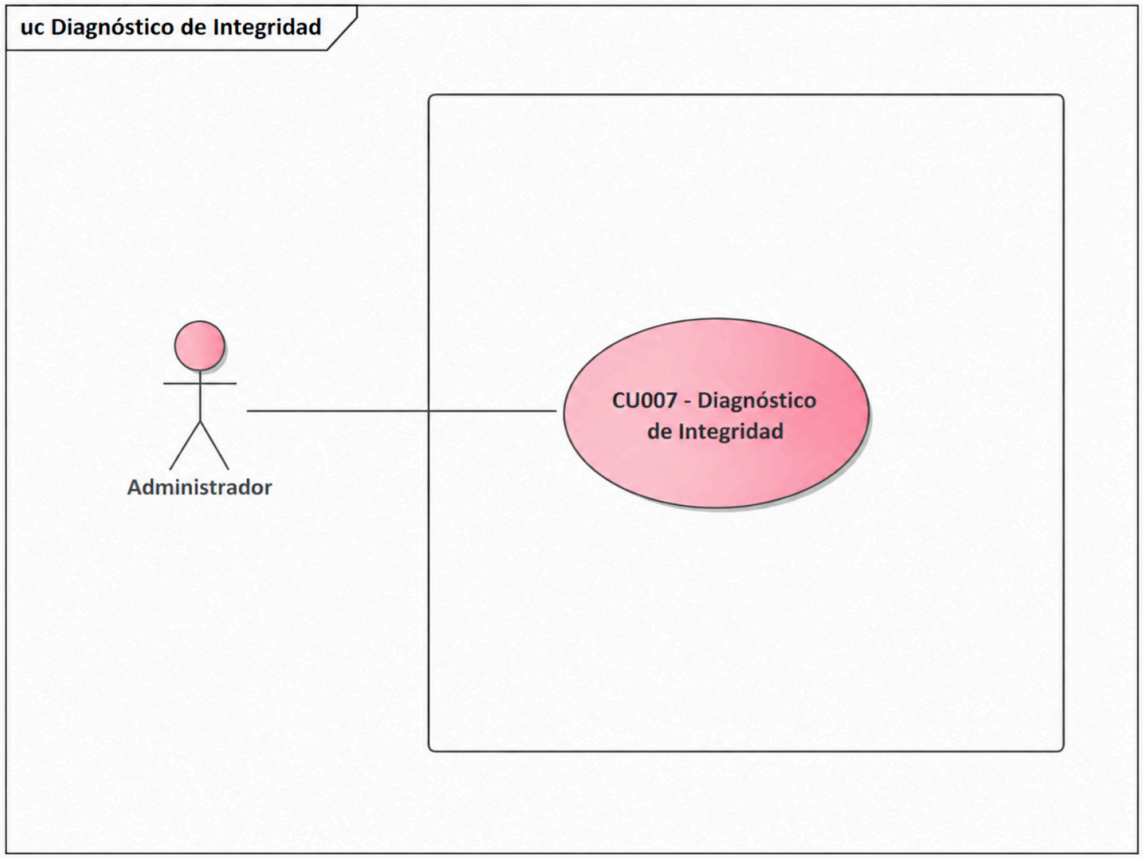
La solución fue desarrollada de manera genérica para permitir su reutilización sobre distintas entidades del sistema sin necesidad de implementar mecanismos específicos para cada una de ellas.

Beneficios de la solución

La implementación de dígitos verificadores permite:

- Detectar modificaciones realizadas directamente sobre la base de datos.
- Identificar inconsistencias en la información almacenada.
- Proteger entidades críticas del sistema.
- Incrementar la confiabilidad y seguridad de los datos.
- Garantizar controles de integridad antes del acceso de los usuarios a la aplicación.

Caso de Uso



Especificación de Caso de Uso

ID Y NOMBRE:	CU 007 - Diagnóstico de Integridad
DESCRIPCIÓN: Permite al Administrador verificar que los datos de las tablas críticas de la base de datos no hayan sido modificados directamente (fuera de la aplicación), detectar qué filas presentan inconsistencias y repararlas o recalcular todos los dígitos verificadores. El diagnóstico también se ejecuta de forma automática cada vez que se inicia la aplicación.	

ACTOR PRINCIPAL: Administrador

ACTORES SECUNDARIO: -

PRECONDICIONES:

- El usuario debe tener una sesión activa con permiso de gestión de usuarios.
- Las tablas de dígitos verificadores deben estar creadas en la base de datos.

PUNTOS DE EXTENSIÓN: -

PUNTOS DE INCLUSIÓN: -

8. **ESCENARIO PRINCIPAL:**
9. 1. El usuario accede a Administrar → Diagnóstico de Integridad desde el menú principal.
10. 2. El sistema calcula y compara los dígitos verificadores almacenados con los valores actuales de cada fila y de cada tabla protegida.
11. 3. El sistema muestra el resultado en la pestaña "Diagnóstico": estado general (ÍNTEGRO o COMPROMETIDO), valores de dígito verificador almacenado y calculado, y listado de filas con inconsistencias resaltadas en rojo.
12. 4. El sistema muestra en la pestaña "Historial de Verificaciones" el registro de las últimas verificaciones realizadas, con el detalle de la causa de cada resultado.
13. 5. Si todo está íntegro, el sistema confirma que no hay problemas de integridad.
14. 6. Si hay inconsistencias, el usuario decide recalcular todos los dígitos o reparar las filas afectadas (ver flujos alternativos).
15. 7. El usuario puede presionar "Actualizar" en cualquier momento para ejecutar el diagnóstico nuevamente.

FLUJOS ALTERNATIVOS:

6.1 El usuario presiona "Recalcular Todo": el sistema muestra una advertencia indicando que se sobreescribirán todos los dígitos verificadores almacenados. El usuario confirma e ingresa sus credenciales de Administrador. Si las credenciales son correctas, el sistema

recalcula los dígitos de todas las filas y tablas protegidas, actualiza la base de datos y registra la operación en el historial. Si las credenciales son incorrectas, la operación se cancela.

6.2 El usuario presiona "Reparar Seleccionadas": el sistema solicita las credenciales de Administrador. Si son correctas, abre el asistente de reparación que permite revisar y corregir los datos de las filas afectadas antes de recalcular sus dígitos verificadores.

POSTCONDICIONES:

- Si se recalculó o reparó: todos los dígitos verificadores están actualizados en la base de datos y el estado del diagnóstico es ÍNTEGRO.
- Cada verificación ejecutada queda registrada en el historial de integridad con su resultado, fecha y origen (Arranque, Timer o Manual).
- Si quedaron inconsistencias sin reparar, el estado COMPROMETIDO persiste hasta la próxima corrección.

Diagrama de clases

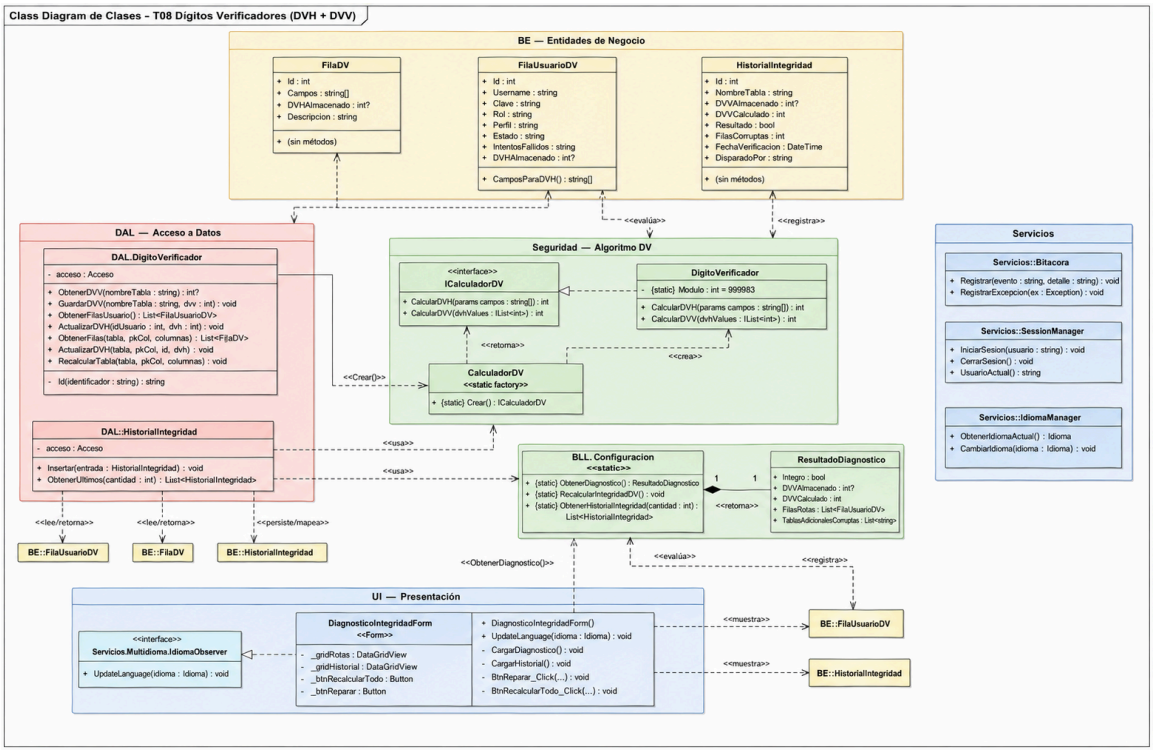
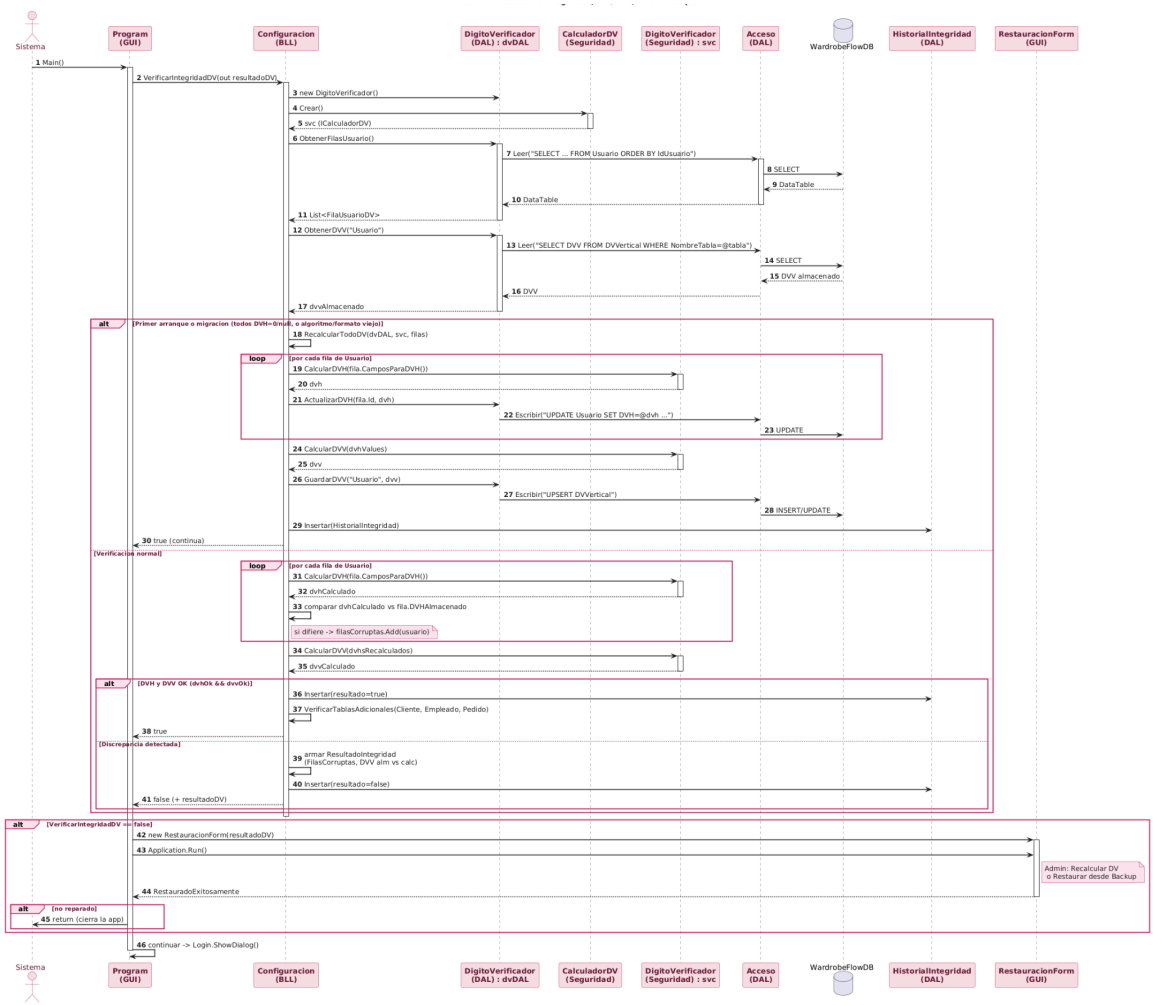
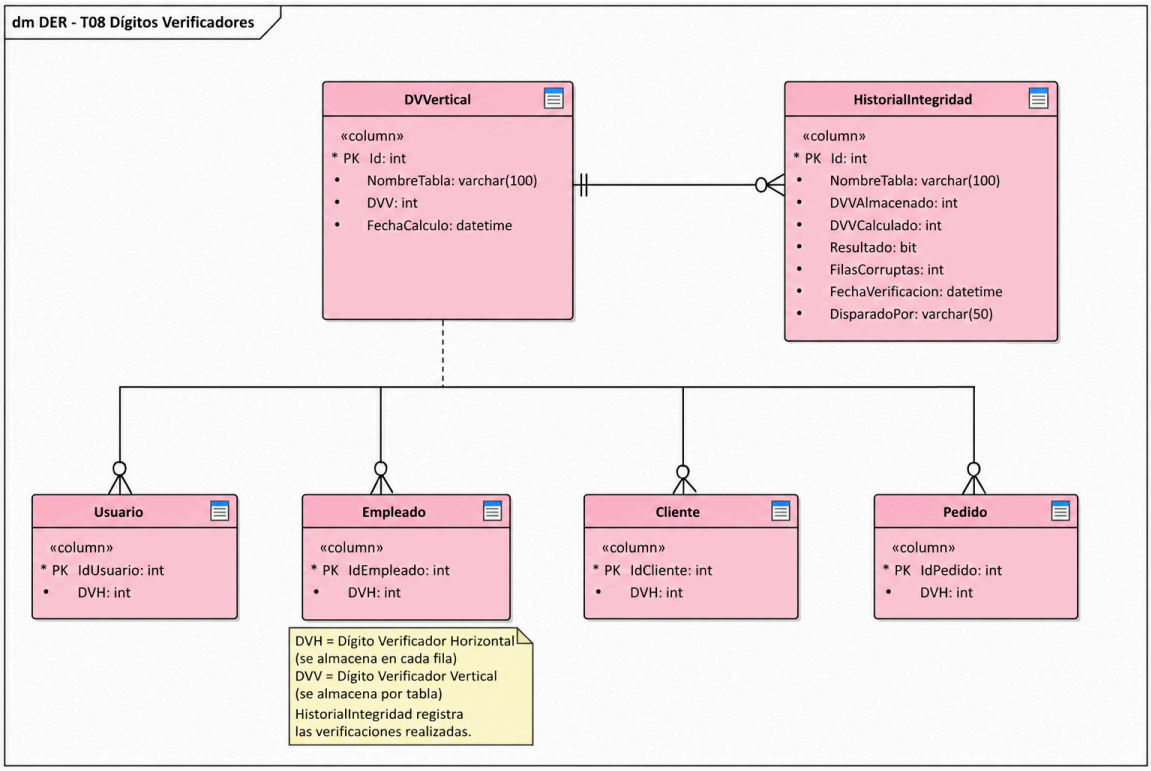


Diagrama de secuencia



DER


dm DER - T08 Dígitos Verificadores
DVVertical
«column»
* PK Id: int
• NombreTabla: varchar(100)
• DVV: int
• FechaCalculo: datetime
HistorialIntegridad
«column»
* PK Id: int
• NombreTabla: varchar(100)
• DVVAlmacenado: int
• DVVCalculado: int
• Resultado: bit
• FilasCorruptas: int
• FechaVerificacion: datetime
• DisparadoPor: varchar(50)
Usuario
«column»
* PK IdUsuario: int
• DVH: int
Empleado
«column»
* PK IdEmpleado: int
• DVH: int
Cliente
«column»
* PK IdCliente: int
• DVH: int
Pedido
«column»
* PK IdPedido: int
• DVH: int
DVH = Dígito Verificador Horizontal
(se almacena en cada fila)
DVV = Dígito Verificador Vertical
(se almacena por tabla)
HistorialIntegridad registra
las verificaciones realizadas.

N00. Procesos de Negocio

WardrobeFlow opera sobre un conjunto de procesos de negocio interrelacionados que cubren el ciclo completo del servicio de alquiler de prendas por suscripción, desde el registro del cliente y la activación de su suscripción hasta la devolución, inspección, registro de incidencias, compra definitiva de unidades y los controles de seguridad, auditoría y reporte operativo.

Todos los procesos son ejecutados íntegramente por el personal interno de WardrobeFlow. El cliente no accede al sistema en ningún momento; su participación se limita a interacciones externas (teléfono, correo electrónico o WhatsApp). Los procesos se encuentran alineados con los módulos M01 a M07 y respetan las restricciones de acceso impuestas por los roles definidos.

N01. Especificación funcional por proceso de negocio

N01. Proceso de Negocio 1 – Gestión de Clientes y Suscripciones

Registrar y mantener los datos de los clientes del servicio, vinculándolos a un plan de suscripción activo que determine el cupo máximo de prendas simultáneas habilitadas para el alquiler.

Roles Intervinientes

Rol	Descripción
Vendedor	Responsable exclusivo del alta, modificación y asignación de planes de suscripción a los clientes. Opera como único punto de contacto entre el cliente externo y el sistema.
Administrador	Acceso de supervisión y configuración general. Puede intervenir ante bloqueos o inconsistencias en los registros.

Descripción funcional del proceso

El Vendedor registra al cliente en el sistema ingresando sus datos de contacto (nombre, apellido, correo electrónico y teléfono) y le asigna un plan de suscripción activo. El sistema valida que el cliente no posea un plan vigente previo; en caso contrario, permite la modificación del plan existente. El cliente es una entidad completamente externa al sistema y no dispone de credenciales de acceso en ningún momento: toda interacción con el sistema se realiza a través del personal interno. Una vez registrado, el cliente recibe un identificador único y el cupo habilitado queda disponible para la generación de pedidos de alquiler.

Diagrama de Proceso

Diagrama de Clases

N02. Proceso de Negocio 2 – Gestión de Inventario de Prendas

Administrar el ciclo de vida completo de cada unidad física del catálogo de prendas, garantizando trazabilidad individual y disponibilidad actualizada en tiempo real para la operación del servicio.

Roles Intervinientes

Rol	Descripción
Controlador de Stock	Gestiona el ciclo de vida físico del catálogo: alta, modificación de atributos y control de estados de cada unidad.
Vendedor	Acceso de consulta en tiempo real para verificar disponibilidad antes de generar pedidos.

Descripción funcional del proceso

El Controlador de Stock registra cada prenda ingresando sus atributos físicos (talle, color, categoría y valor de reposición) a través del módulo de Catálogo de

Prendas. El sistema asigna automáticamente un identificador único a cada unidad para su trazabilidad individual. El estado inicial obligatorio al registrar una prenda es "Disponible". A lo largo del ciclo operativo, cada unidad puede transitar por los estados "Disponible", "En uso", "En Limpieza" y "Baja". El sistema impide asignar prendas cuyo estado sea distinto de "Disponible". El Vendedor puede consultar el catálogo en tiempo real para verificar disponibilidad antes de generar un pedido; cuando una prenda figura como "En uso", el sistema expone el detalle del cliente que la posee actualmente.

Diagrama de Proceso

Diagrama de Clases

N03. Proceso de Negocio 3 – Gestión de Pedidos de Alquiler

Generar y confirmar pedidos de alquiler en nombre del cliente, validando en tiempo real la disponibilidad de las prendas seleccionadas y el cupo vigente del plan de suscripción antes de la confirmación.

Roles Intervinientes

Rol	Descripción
Vendedor	Genera y confirma pedidos de alquiler en nombre del cliente, validando disponibilidad y cupos antes de confirmar.

Descripción funcional del proceso

El Vendedor genera un pedido seleccionando prendas del catálogo para un cliente determinado. Antes de confirmar, el sistema verifica que cada prenda se encuentre en estado "Disponible" y que la cantidad solicitada no supere el cupo máximo del plan activo del cliente. Un cliente con prendas en circulación no puede generar un nuevo pedido hasta devolver las unidades del ciclo anterior. Al confirmar el pedido, el sistema actualiza automáticamente el estado de cada

prenda seleccionada de "Disponible" a "En uso" y genera un número de pedido único que queda disponible para su procesamiento por el módulo de despacho.

Diagrama de Proceso

Diagrama de Clases

N04. Proceso de Negocio 4 – Gestión de Despachos

Registrar el estado del despacho de los pedidos confirmados y notificar automáticamente al cliente el envío con los datos de seguimiento correspondientes.

Roles Intervinientes

Rol	Descripción
Operador de Inventario	Único rol habilitado en el módulo M05. Verifica pedidos, registra el estado del despacho y confirma el envío al cliente.

Descripción funcional del proceso

El Operador de Inventario accede al módulo de Despacho, que presenta exclusivamente los pedidos confirmados pendientes de envío. Esta restricción es deliberada para evitar la exposición de información ajena a su función. El operador verifica los datos del pedido, prepara el paquete y registra el estado del despacho (Despachado, En curso u On Hold). Al confirmar, el sistema envía automáticamente una notificación por correo electrónico al cliente con el número de pedido y los datos de seguimiento. El evento queda asentado en la bitácora del sistema.

Diagrama de Proceso

Diagrama de Clases

N05. Proceso de Negocio 5 – Gestión de Devoluciones

Registrar el reingreso de prendas devueltas al catálogo activo, liberar el cupo del cliente en su suscripción y derivar al flujo de incidencias aquellas unidades que presenten anomalías en la inspección.

Roles Intervinientes

Rol	Descripción
Controlador de Stock	Recibe físicamente las prendas devueltas, realiza la inspección y registra la devolución en el sistema.
Supervisor	Interviene cuando la inspección detecta anomalías que requieren tratamiento como incidencia.

Descripción funcional del proceso

Al concluir el ciclo de alquiler, el Controlador de Stock recibe físicamente las prendas devueltas y registra la devolución vinculando cada unidad al pedido de origen. Si la prenda se encuentra en condiciones adecuadas, el sistema aprueba el reingreso, actualiza su estado a "Disponible" y libera de forma inmediata el cupo correspondiente en la suscripción del cliente. Si durante la inspección se detectan anomalías como daños, manchas o extravíos, el sistema redirige automáticamente la operación al flujo de gestión de incidencias para su tratamiento y eventual cálculo de cargos adicionales.

Diagrama de Proceso

Diagrama de Clases

N06. Proceso de Negocio 6 – Gestión de Incidencias

Registrar y gestionar las anomalías detectadas en prendas devueltas, calcular automáticamente los cargos adicionales correspondientes y someter su aplicación a la autorización jerárquica del Supervisor.

Roles Intervinientes

Rol	Descripción
Controlador de Stock	Registra la incidencia documentando el tipo y gravedad de la anomalía detectada durante la inspección.
Supervisor	Revisa la incidencia, evalúa la documentación y autoriza o rechaza el cargo adicional calculado por el sistema.

Descripción funcional del proceso

Cuando una prenda devuelta presenta anomalías, el Controlador de Stock registra la incidencia en el sistema documentando el tipo de daño (rotura, mancha, extravío, devolución tardía u otro). El sistema calcula automáticamente el cargo adicional en función del tipo de incidencia y el valor de reposición de la prenda. El cargo no se aplica de manera automática: requiere la revisión y autorización explícita del Supervisor, quien evalúa la documentación registrada antes de aprobar o rechazar el cobro. Únicamente tras la aprobación jerárquica el cargo queda confirmado, la incidencia pasa al estado "Resuelta" y el evento queda asentado en la bitácora del sistema.

Diagrama de Proceso

Diagrama de Clases

N07. Proceso de Negocio 7 – Compra Definitiva de Prenda

Gestionar la adquisición definitiva de una prenda en alquiler por parte del cliente, registrando la operación de compra-venta y ejecutando la baja lógica de la unidad del catálogo activo.

Roles Intervinientes

Rol	Descripción
Vendedor	Gestiona íntegramente la operación de compra-venta desde el sistema en nombre del cliente, registrando la venta y ejecutando la baja lógica de la prenda.

Descripción funcional del proceso

Cuando un cliente desea conservar permanentemente una prenda en alquiler, el Vendedor gestiona la operación de compra-venta desde el sistema. El proceso es manual y está sujeto a disponibilidad y oferta de la organización. El Vendedor registra el precio de venta acordado, el sistema genera el comprobante correspondiente y ejecuta simultáneamente la baja lógica de la prenda: su estado pasa a "Baja" y la unidad queda retirada de forma definitiva del circuito de alquiler, sin posibilidad de reactivación posterior. El evento de venta queda asentado en la bitácora del sistema.

Diagrama de Proceso

Diagrama de Clases

N08. Proceso de Negocio 8 – Generación de Reportes Operativos

Generar reportes de gestión operativa consolidados y configurables por parámetros, disponibles para visualización en pantalla y exportación en formato PDF para archivo o distribución interna.

Roles Intervinientes

Rol	Descripción
Supervisor	Único rol habilitado para configurar, generar y exportar reportes de gestión desde el módulo M07.

Descripción funcional del proceso

El Supervisor accede al módulo de Reportes y configura el tipo de reporte deseado (estado del stock, pedidos por período, rendimiento por vendedor, incidencias, entre otros) junto con el rango de fechas y los criterios de filtrado correspondientes. El sistema consolida datos provenientes de los módulos de pedidos, inventario, despachos e incidencias para producir métricas operativas integradas y actualizadas al momento de la consulta. Los reportes pueden visualizarse en forma de dashboard dentro de la aplicación y exportarse en formato PDF para su archivo o distribución interna.

Diagrama de Proceso

Diagrama de Clases